VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 3: Tasks, Functions and Procedural Concepts

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 3 Overview 3

Part 1: Procedural Blocks and Assignments 4
- 1.1 Procedural Blocks: initial, always and final 4
- 1.2 Statement Blocks: begin-end, Named Blocks and Local Variables 8
- 1.3 Blocking and Nonblocking Assignments 11
- 1.4 The Simulation Event Regions 15
- 1.5 Procedural Statements at a Glance 18

Part 2: Functions 20
- 2.1 Why Subroutines Exist 20
- 2.2 Functions in SystemVerilog 20
- 2.3 Returning Values and Void Functions 24
- 2.4 Why a Function Cannot Consume Simulation Time 27

Part 3: Tasks 31
- 3.1 Tasks in SystemVerilog 31
- 3.2 Timing Control Inside Tasks 35
- 3.3 Differences Between Functions and Tasks 38
- 3.4 Tasks Calling Other Tasks and Functions 40

Part 4: Arguments 45
- 4.1 Argument Directions: input, output, inout and ref 45
- 4.2 Pass by Value 49
- 4.3 Pass by Reference Using ref 52
- 4.4 Default Argument Values and Binding by Name 56
- 4.5 Passing Arrays, Strings and Structs 59

Part 5: Lifetime, Recursion and Class Methods 62
- 5.1 Automatic and Static Lifetime 62
- 5.2 The Shared-Variable Bug in Static Tasks 66
- 5.3 Recursive Functions 69
- 5.4 Function and Task Declarations Inside Classes 72

Part 6: Module 3 Revision and Practice 77
- 6.1 Revision Summary 77
- 6.2 Interview Questions and Answers 79
- 6.3 Output-Prediction Questions 82
- 6.4 Find-the-Error and Debugging Questions 85
- 6.5 Coding Exercises with Solutions 90
- 6.6 Concepts You Must Understand Before Moving Forward 95
- 6.7 What Comes Next 96

Module 3 Overview

Module 1 covered the data that a SystemVerilog program stores. Module 2 covered collections of that data. This module covers **behaviour**: how statements are executed, how they are grouped into reusable subroutines (functions and tasks), how data is passed in and out of those subroutines, and how long the variables inside them live.

Everything in a class-based testbench is built from these pieces. A driver is a class whose main method is a task. A scoreboard's reference model is a function. A transaction's `print`, `copy` and `compare` are functions. If you are unsure why one is a task and the other a function, or why a value did not come back from a call, the answer is in this module.

What this module covers

Part	Content
Part 1	Procedural blocks (`initial`, `always`, `always_comb`, `always_ff`, `always_latch`, `final`), statement blocks, blocking and nonblocking assignments, the simulation event regions
Part 2	Functions: declaration, returning values, `void` functions, why a function cannot consume simulation time
Part 3	Tasks: declaration, timing control inside tasks, functions versus tasks, subroutines calling other subroutines
Part 4	Arguments: `input`, `output`, `inout`, `ref`, `const ref`, pass by value, pass by reference, default values, binding by name
Part 5	Lifetime (`static` and `automatic`), the shared-variable bug, recursion, functions and tasks inside classes
Part 6	Revision summary, interview questions, output-prediction, find-the-error, coding exercises

What you need from earlier modules

- Module 1: two-state and four-state types, `logic` and `int`, signedness, expression width, `typedef`, enums, the ALU design (`alu_pkg`, `alu`).
- Module 2: fixed-size arrays, dynamic arrays and queues at the level of declaring them and using `foreach`.

Conventions. The naming is the same as in the other modules: demonstration modules are called `tb_<topic>`, exercises `exNN_<name>`, the DUT is the 8-bit `alu`, and the transaction class is `alu_txn`. Every example with a file name is extracted to `examples/module-03/`.

> **Note:** The "Expected output" blocks were worked out by hand from the IEEE 1800 rules; they were not captured from a simulator run. Where the standard leaves an order undefined (for example which of two parallel processes starts first), the text says so. Run each example yourself.

Part 1: Procedural Blocks and Assignments

1.1 Procedural Blocks: initial, always and final

In plain English

SystemVerilog code is of two kinds.

- **Declarations and structure**: "this signal exists", "this module contains that module", "this wire always equals `a & b`". These statements describe things that exist for the whole simulation.
- **Procedural code**: "do this, then do that, then wait for the clock, then do something else". These statements are executed one after another, like a program in C.

Procedural statements cannot be written just anywhere inside a module. They must be placed inside a **procedural block**, which tells the simulator when to start executing them and whether to repeat them.

Technical definition

A procedural block is a construct that creates one **process** (also called a thread). Each process executes its statements sequentially. All processes in the design run concurrently with each other.

Block	Starts	Repeats?	Synthesizable	Purpose
`initial`	At time 0	No, runs once	No (ignored by ASIC synthesis)	Testbench stimulus, initialisation, starting the test
`always`	At time 0	Yes, forever	Yes, when written in a synthesizable style	General-purpose; Verilog legacy
`always_comb`	At time 0, then on any change of an input	Yes	Yes	Combinational logic
`always_ff`	On the specified clock/ reset edge	Yes	Yes	Flip-flops
`always_latch`	At time 0, then on any change of an input	Yes	Yes	Latches (intended ones)
`final`	When the simulation ends	No, runs once	No	End-of-test reports

A module can contain any number of each. There is **no defined order** between different blocks that become active at the same simulation time. Two `initial` blocks both start at time 0, and the standard does not say which executes first.

Why they are needed

Hardware is concurrent: every gate and flip-flop operates at the same time. A simulator runs on a sequential computer, so it models concurrency with many small processes that each run for a moment and then wait. Procedural blocks are how you create those processes. `always` blocks model hardware that reacts to its inputs for ever. `initial` blocks model the one-off sequence of actions that a test performs.

How it works internally

```
Time 0:  every initial and always block becomes a process and is put on the
         simulator's "ready" list.

         +-----------+     runs until it reaches a timing control     +-----------+
         |  process  | ---------------------------------------------> | suspended |
         |  running  |      ( #delay,  @(event),  wait(cond) )        |  waiting  |
         +-----------+ <--------------------------------------------- +-----------+
                          the delay expires / the event occurs

- Only one process actually executes at any instant.
- A process is never interrupted: it runs until IT reaches a timing control or ends.
- When no process is ready at the current time, simulation time advances to the
  next scheduled event.
- initial : when the last statement finishes, the process ends for good.
- always  : when the last statement finishes, execution jumps back to the first.
```

The second point matters. Because a process runs without interruption until it suspends itself, an `always` block that contains **no timing control** never suspends, never lets time advance, and hangs the simulation at time 0.

Syntax

```
initial begin
  // statements, executed once
end

always @(posedge clk) begin        // general always with an event control
  // statements, executed at every rising edge
end

always #5 clk = ~clk;              // general always with a delay: clock generator

always_comb begin                  // no sensitivity list: it is inferred
  // combinational logic
end

always_ff @(posedge clk or negedge rst_n) begin
  // sequential logic, nonblocking assignments
end

always_latch begin
  if (en) q <= d;                  // intended latch
end

final begin
  // statements executed in zero time when simulation finishes
end
```

Complete example

tb_procedural_blocks.sv

```
module tb_procedural_blocks;

  logic clk = 0;
  int   count;

  always #5 clk = ~clk;

  initial begin
    $display("[%0t] initial block 1 starts", $time);
    #12;
    $display("[%0t] initial block 1 ends", $time);
  end

  initial begin
    #3;
    $display("[%0t] initial block 2 runs in parallel", $time);
  end

  always @(posedge clk) begin
    count++;
    $display("[%0t] always block, count=%0d", $time, count);
  end

  initial #27 $finish;

  final $display("[%0t] final block, count=%0d", $time, count);

endmodule
```

EXPECTED OUTPUT

```
[0] initial block 1 starts
[3] initial block 2 runs in parallel
[5] always block, count=1
[12] initial block 1 ends
[15] always block, count=2
[25] always block, count=3
[27] final block, count=3
```

Line by line

Line	Explanation
`always #5 clk = ~clk;`	A process that waits 5 time units, inverts `clk`, and repeats for ever. Rising edges occur at 5, 15, 25...
First `initial`	Starts at time 0, prints, suspends for 12 time units, prints again and ends.
Second `initial`	Also starts at time 0. It is a separate process, so its `#3` runs in parallel with the `#12` of the first block. Its message appears at time 3, between the two messages of block 1.
`always @(posedge clk)`	Suspends until a rising edge, executes the body, then returns to the top and suspends again.
`initial #27 $finish;`	A third `initial` block with a single statement. `begin` ... `end` is needed only for more than one statement.
`final`	Runs once when `$finish` is executed (or when no events remain). The time is still 27.

The always family compared

Property	`always @*`	`always_comb`	`always_ff`	`always_latch`
Sensitivity list	Inferred from signals read in the block	Inferred, **including** signals read inside called functions	Written explicitly	Inferred
Executes once at time 0	No (waits for an input change)	Yes	No	Yes
Same variable written by another process	Allowed	Error	Error	Error
Timing controls inside the body	Allowed	Not allowed	Only the one at the top	Not allowed
Tool checks the intent	No	Warns if a latch is inferred	Requires sequential logic	Requires latch behaviour

initial versus always versus final

Property	`initial`	`always`	`final`
Starts	Time 0	Time 0	End of simulation
Executions	One	Unlimited	One
May contain delays and event controls	Yes	Yes (must contain at least one)	No, it must execute in zero time
Number allowed per module	Any	Any	Any
Order among several of the same kind	Undefined	Undefined	Undefined
Typical use	Stimulus, starting the test	Clocks, RTL, monitors	Summary of pass/fail counts, coverage

Interview trap: "Two `initial` blocks in one module: which runs first?" The standard does not define it. Most simulators run them in source order, but code must never depend on that. If the order matters, put the statements in one block or synchronise them explicitly.

Common mistake: Writing `always clk = ~clk;` without a delay. The process never suspends, so simulation time cannot advance and the simulator hangs at time 0 (some tools report a zero-delay loop).

Common mistake: Expecting a `final` block to run when a simulation is killed or stopped by a fatal run-time crash. It runs on a normal end: an explicit `$finish`, or when the event queue becomes empty. A `final` block behaves like a function: it cannot contain `#`, `@` or `wait`.

Verification application: `tb_top` normally contains one `always` for the clock, one `initial` for reset, and one `initial` that creates the test object and calls its `run()` task (Module 9). The scoreboard's totals are printed either at the end of the test's `run()` task or from a `final` block, so that they appear even when the test ends through `$finish` elsewhere.

Interview questions

Q1. What is the difference between `initial` and `always`? Both start at time 0. An `initial` block executes its statements once and terminates. An `always` block returns to its first statement after the last one and repeats for the whole simulation.

Q2. Is `initial` synthesizable? Not for ASIC synthesis, which ignores it. Some FPGA tools use it to set power-up values of registers and memory contents.

Q3. What happens with an `always` block that has no timing control? It becomes a zero-delay infinite loop. Simulation time never advances and the simulation hangs.

Q4. What is a `final` block? A block that executes once at the end of simulation, in zero time. It may not contain delays, event controls or task calls that consume time. It is used to print summaries.

Q5. Why prefer `always_comb` to `always @*`? It executes once at time 0 so the outputs are consistent with the inputs from the start, its sensitivity includes signals read in called functions, it forbids other processes from writing the same variables, and tools check that the result is really combinational.

Q6. How many `initial` blocks can a module have, and in what order do they execute? Any number. They all start at time 0 as separate concurrent processes; the order of execution within that time step is not defined.

1.2 Statement Blocks: begin-end, Named Blocks and Local Variables

In plain English

Wherever SystemVerilog expects one statement (the body of an `if`, a loop, an `always`), you can supply several by wrapping them in `begin` and `end`. It plays the role of the braces `{ }` in C.

Technical definition

A **sequential block** `begin` ... `end` groups statements that are executed one after another, in order. A **parallel block** `fork` ... `join` groups statements that are executed concurrently, each as its own process (Module 7). Either kind of block may be given a **name**, and may declare its own local variables.

Syntax

```
begin : block_name          // the name is optional
  int local_var;            // declarations first
  statement_1;
  statement_2;
end : block_name            // the closing name is optional and must match
```

What a block name gives you

Feature	Use
Readability	A long `end` can be matched to its `begin`
`disable block_name;`	Abandons execution of that block and continues after its `end`

Feature	Use
Hierarchical access	A variable declared inside is reachable as `block_name.local_var` in waveforms and debug
Coverage and assertion reports	Named blocks appear by name

Local variables and the initialiser rule

A variable may be declared at the start of any `begin`...`end` block. It is visible only inside that block.

There is a rule here that produces a compile error which confuses beginners. Inside a **static** context (an `initial` or `always` block in a module, or a task or function that is not `automatic`), a local variable is static: one copy exists for the whole simulation. If such a declaration has an initial value, the standard requires you to write the keyword `static` or `automatic` explicitly, so that it is clear whether the initialisation happens once or on every entry.

```
initial begin
  int a;                  // legal: static, starts at 0
  // int b = 5;           // not legal in strict IEEE 1800: lifetime keyword required
  static    int c = 5;    // legal: initialised once, before time 0
  automatic int d = 5;    // legal: initialised each time the block is entered
  a = 5;                  // the simplest alternative: declare, then assign
end
```

Simulators differ in how strictly they apply this: some reject the commented line, others accept it with a warning. Section 5.1 explains lifetime in full. A loop variable declared in the loop header, `for (int i = 0; ...)`, is always automatic and needs no keyword.

Complete example

tb_blocks.sv

```
module tb_blocks;

  int data [5] = '{3, 8, 12, 7, 20};

  initial begin : main_blk
    int found_at;
    found_at = -1;

    for (int i = 0; i < 5; i++) begin : scan
      if (data[i] > 10) begin
        found_at = i;
        break;
      end
    end : scan
    $display("1. first value above 10 is at index %0d", found_at);

    begin : guarded
      $display("2. inside guarded block");
      if (found_at == 2) disable guarded;
      $display("   this line is skipped");
    end : guarded
    $display("3. after guarded block");
  end : main_blk

endmodule
```

EXPECTED OUTPUT

```
1. first value above 10 is at index 2
2. inside guarded block
3. after guarded block
```

Line by line

Line	Explanation
`initial begin : main_blk`	A named sequential block.
`int found_at;` then `found_at = -1;`	Declared without an initialiser and assigned separately, which avoids the lifetime-keyword rule described above.
`begin : scan`	The loop body is itself a named block.
`break;`	Leaves the nearest enclosing loop. `data[2]` is 12, the first value above 10.
`disable guarded;`	Terminates the block named `guarded`. Execution continues after `end : guarded`, so the next `$display` is skipped.

Ways to leave code early

Statement	Effect	Where it can be used
`break;`	Leaves the innermost loop	Inside loops
`continue;`	Skips to the next iteration of the innermost loop	Inside loops
`return;` / `return value;`	Leaves the current task or function	Inside tasks and functions
`disable name;`	Terminates the named block or task, including every process started inside it	Anywhere the name is visible
`disable fork;`	Terminates all processes spawned by the current process	Module 7

`break`, `continue` and `return` were added by SystemVerilog. In Verilog the only way to do any of these was `disable`.

> **Common mistake:** Forgetting `begin` … `end` around a multi-statement body. In `if (x) a = 1; b = 2;` only the first assignment belongs to the `if`; `b = 2` always executes. Indentation means nothing to the compiler.

Interview questions

Q1. What is the difference between `begin` … `end` and `fork` … `join`? Statements in `begin` … `end` execute sequentially; a delay in one statement postpones all the following ones. Statements in `fork` … `join` start at the same time as separate processes; the block finishes when all of them have finished.

Q2. What is a named block and why use it? A `begin` … `end` or `fork` … `join` with a label. The name allows the block to be disabled, lets its local variables be seen hierarchically, and documents the code.

Q3. What is the difference between `break` and `disable`? `break` leaves the innermost loop only. `disable` terminates any named block or task, wherever it is called from, and also kills processes that were started inside that block.

1.3 Blocking and Nonblocking Assignments

In plain English

SystemVerilog has two assignment operators for procedural code.

- `=` is the **blocking** assignment: "do it now". The left-hand side has its new value before the next statement is executed. It is the ordinary assignment of a programming language.
- `<=` is the **nonblocking** assignment: "work out the value now, but hand it over later, at the end of this time step". Statements that follow in the same block still see the old value.

The second form exists to model flip-flops. All flip-flops clocked by the same edge capture their inputs at the same instant; none of them sees the new output of another. Nonblocking assignment reproduces that: first every right-hand side is evaluated using old values, then every left-hand side is updated.

Technical definition

	Blocking `=`	Nonblocking `<=`
Step 1	Evaluate the right-hand side	Evaluate the right-hand side
Step 2	Update the left-hand side immediately, in the Active region	Schedule the update for the NBA (nonblocking assignment) region of the same time step
Next statement sees	The new value	The old value
"Blocks" the next statement	Yes: the next statement cannot run until the assignment is complete	No

How it works internally

```
c = 1;  d = 2;                          c      d
------------------------------------    -----  -----
c <= d;     RHS evaluated: 2  (saved)     1      2
d <= c;     RHS evaluated: 1  (saved)     1      2
...rest of the active processes run...    1      2
--- NBA region of this time step ---
apply saved updates: c = 2, d = 1         2      1      values swapped

a = 1;  b = 2;                          a      b
------------------------------------    -----  -----
a = b;      a becomes 2 immediately       2      2
b = a;      b = new a = 2                 2      2      value 1 is lost
```

Complete example

tb_blocking_nonblocking.sv

```
module tb_blocking_nonblocking;

  int a, b, c, d;

  initial begin
    a = 1;  b = 2;
    a = b;
    b = a;
    $display("1. blocking            : a=%0d b=%0d", a, b);

    c = 1;  d = 2;
    c <= d;
    d <= c;
    $display("2. nonblocking, before : c=%0d d=%0d", c, d);
    #1;
    $display("3. nonblocking, after  : c=%0d d=%0d", c, d);
  end

endmodule
```

EXPECTED OUTPUT

```
1. blocking            : a=2 b=2
2. nonblocking, before : c=1 d=2
3. nonblocking, after  : c=2 d=1
```

Line by line

Line	Explanation
`a = b; b = a;`	`a` becomes 2 at once. The second statement then copies the new `a`. Both end up 2.
`c <= d; d <= c;`	Both right-hand sides are read immediately (2 and 1), but neither variable changes yet.
Line 2 of the output	The `$display` runs in the same Active region, before the NBA region, so it still shows the old values.
`#1;`	Lets the time step finish. The NBA updates have been applied: the values are swapped.

The reason nonblocking exists: a pipeline

tb_pipeline.sv

```
module tb_pipeline;

  logic     clk = 0;
  bit [3:0] d = 4'd5;
  bit [3:0] q1, q2, q3;      // written with nonblocking assignments
  bit [3:0] b1, b2, b3;      // written with blocking assignments

  always #5 clk = ~clk;

  always_ff @(posedge clk) begin
    q1 <= d;
    q2 <= q1;
    q3 <= q2;
  end

  always @(posedge clk) begin
    b1 = d;
    b2 = b1;
    b3 = b2;
  end

  initial begin
    repeat (3) begin
      @(negedge clk);
      $display("[%0t] nonblocking q1 q2 q3 = %0d %0d %0d    blocking b1 b2 b3 = %0d %0d %0d",
               $time, q1, q2, q3, b1, b2, b3);
    end
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[10] nonblocking q1 q2 q3 = 5 0 0    blocking b1 b2 b3 = 5 5 5
[20] nonblocking q1 q2 q3 = 5 5 0    blocking b1 b2 b3 = 5 5 5
[30] nonblocking q1 q2 q3 = 5 5 5    blocking b1 b2 b3 = 5 5 5
```

With nonblocking assignments the value 5 moves one stage per clock, which is what three flip-flops in series do. With blocking assignments the value passes through all three variables in a single clock edge: the code describes one flip-flop with three names, not a three-stage pipeline.

Races

When two processes run at the same time step, their order is undefined. With blocking assignments this causes a **race condition**:

```
always @(posedge clk) x = y;      // process A
always @(posedge clk) y = x;      // process B
```

If A runs first, both variables end up with the old `y`. If B runs first, both end up with the old `x`. The result depends on the simulator. With `<=` in both blocks, the right-hand sides are read before either variable changes, so the values are swapped whichever process runs first.

Coding guidelines

Rule	Reason
Sequential logic (`always_ff`): use `<=`	Models simultaneous capture; avoids races between flip-flops
Combinational logic (`always_comb`): use `=`	A later statement in the block must see the result of an earlier one
Never mix `=` and `<=` to the same variable	Unpredictable results, rejected by synthesis
Never assign the same variable from two `always` blocks	Multiple drivers
Testbench stimulus to the DUT at a clock edge: use `<=`, or drive through a clocking block	Avoids a race between the testbench and the DUT (Module 8)
Testbench internal variables (counters, class properties, local variables): use `=`	They are ordinary program variables. Automatic variables may not be assigned with `<=` at all

Comparison table

Property	Blocking `=`	Nonblocking `<=`
Update happens	Immediately	In the NBA region, after all active processes of the time step
Order of statements matters	Yes	No (for the values read)
Models	Combinational logic, software-style variables	Flip-flops, registered outputs
Legal in `assign`	The `=` there is a continuous assignment, a different construct	No
Legal on automatic variables and class properties	Yes	Automatic variables: no. Class properties: yes
Also means	—	"less than or equal" inside an expression

`<=` is an assignment when it appears as a statement and a comparison when it appears inside an expression: in `a <= b <= c;` the first is the assignment and the second is the comparison.

> **Interview trap:** "`x = 1; x <= 2; $display(x);` What is printed, and what is `x` afterwards?" 1 is printed, because the nonblocking update has not yet been applied. At the end of the time step `x` becomes 2.

> **Interview trap:** "Does the nonblocking assignment evaluate its right-hand side later?" No. The right-hand side is evaluated immediately, when the statement executes. Only the update of the left-hand side is postponed.

Interview questions

Q1. What is the difference between blocking and nonblocking assignments? A blocking assignment evaluates the right-hand side and updates the left-hand side immediately, before the next statement executes. A nonblocking assignment evaluates the right-hand side immediately but updates the left-hand side at the end of the current time step, in the NBA region, so other statements in the time step see the old value.

Q2. Why are nonblocking assignments used for sequential logic? All flip-flops on one clock edge must sample their inputs before any of them changes its output. Nonblocking assignment separates sampling from updating, so the result does not depend on the order in which the simulator runs the `always` blocks.

Q3. Write code to swap two variables without a temporary. `a <= b; b <= a;` in the same block (or in two blocks triggered by the same event).

Q4. What happens if blocking assignments are used in a clocked block that models a shift register? Within one block the value ripples through all the stages in one clock, so the stages collapse into one. Between separate blocks the result depends on execution order: a race.

Q5. Can nonblocking assignments be used in an `initial` block? Yes. They are commonly used there to drive DUT inputs at a clock edge without racing with the DUT.

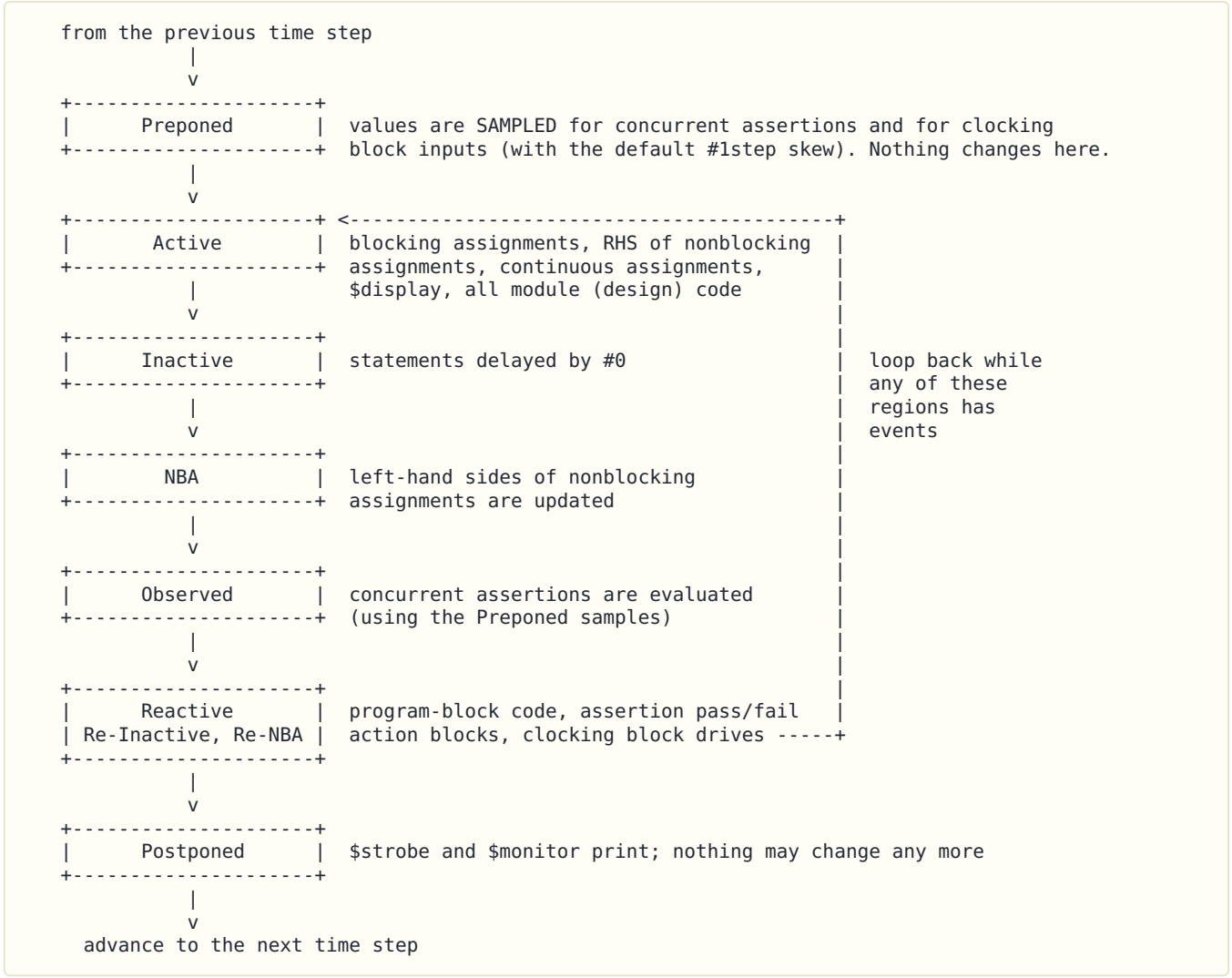
1.4 The Simulation Event Regions

In plain English

Many things are scheduled to happen at the same simulation time: RTL blocks triggered by the clock, testbench code triggered by the same clock, nonblocking updates, assertions, `$monitor` printing. To give predictable results, the simulator divides each time step into ordered phases called **regions** and assigns every kind of activity to one of them.

You do not need to memorise the full algorithm. You do need to know the order of the main regions and what runs in each, because the answers to "why did my testbench sample the old value?" and "why does `$strobe` print something different from `$display`?" are found here, and because clocking blocks and program blocks (Module 8) are defined in terms of these regions.

The regions of one time step



What runs where

Region	Activity
Preponed	Sampling of signals for concurrent assertions and clocking-block inputs. The values seen are those from just before the time step
Active	Blocking assignments; evaluation of the right-hand side of nonblocking assignments; continuous assignments; evaluation of all module code; `$display` and `$write`
Inactive	Statements that were delayed with `#0`
NBA	Update of the left-hand side of nonblocking assignments
Observed	Evaluation of concurrent assertions
Reactive (with Re-Inactive, Re-NBA)	Code in `program` blocks; action blocks of assertions; drives from clocking blocks
Postponed	`$strobe`, `$monitor`; read-only

Two consequences to remember:

1. An NBA update can wake up more Active processes (for example a flip-flop output feeding combinational logic), so the simulator loops Active → Inactive → NBA until nothing is left. These loops within one time step are called **delta cycles**.
2. Design code runs in the Active/NBA set of regions and program-block testbench code runs in the Reactive set, which comes later. This separation is what removes races between the DUT and the testbench (Module 8).

Complete example

`tb_regions.sv`

```
module tb_regions;

  int x;

  initial begin
    x = 1;
    x <= 2;
    $strobe ("3. $strobe   (Postponed region)   : x=%0d", x);
    $display("1. $display  (Active region)      : x=%0d", x);
    #0;
    $display("2. after #0  (via Inactive region): x=%0d", x);
  end

endmodule
```

EXPECTED OUTPUT

```
1. $display  (Active region)      : x=1
2. after #0  (via Inactive region): x=1
3. $strobe   (Postponed region)   : x=2
```

Line by line

Line	Explanation
`x = 1;`	Blocking: `x` is 1 immediately.
`x <= 2;`	The value 2 is saved; the update is scheduled for the NBA region.
`$strobe(...)`	Does not print now. It schedules the printing for the Postponed region, at the very end of the time step.
`$display(...)`	Prints immediately, in the Active region. The NBA update has not happened: 1.
`#0;`	Suspends the process and moves it to the Inactive region. It resumes after all currently active processes, but **before** the NBA region.
Second `$display`	Still 1. `#0` is not enough to see the result of a nonblocking assignment.
`$strobe` output	Printed last, although it appears first in the source. By then the NBA update has been applied: 2.

$display, $write, $strobe and $monitor

Task	Prints when	Region	Notes
`$display`	Immediately when executed	Active (or Reactive, in a program)	Adds a newline
`$write`	Immediately when executed	Same	No newline

Task	Prints when	Region	Notes
`$strobe`	Once, at the end of the current time step	Postponed	Shows final, settled values
`$monitor`	At the end of every time step in which an argument changed	Postponed	Only one `$monitor` is active at a time; `$monitoroff` / `$monitoron` control it

> **Interview trap:** "`a <= 5; $display(a);` prints the old value. How do you print the new one?" Use `$strobe`, or wait for the next time step. `#0` does not work, because the Inactive region comes before the NBA region.

> **Common mistake:** Using `#0` to "fix" a race. It only moves one process later within the Active/Inactive loop. If another piece of code does the same, the race is back. Use nonblocking assignments, clocking blocks or proper synchronisation instead.

Interview questions

Q1. Name the main regions of a simulation time step in order. Preponed, Active, Inactive, NBA, Observed, Reactive (with Re-Inactive and Re-NBA), Postponed.

Q2. In which region are nonblocking assignments updated? The right-hand side is evaluated in the Active region; the left-hand side is updated in the NBA region.

Q3. What is the difference between `$display`, `$strobe` and `$monitor`? `$display` prints at once. `$strobe` prints once at the end of the current time step, after all assignments including nonblocking ones. `$monitor` is registered once and then prints at the end of any time step in which one of its arguments changed.

Q4. What does `#0` do? It suspends the current process and reschedules it in the Inactive region of the same time step, so it resumes after the other currently active processes, but before nonblocking updates are applied.

Q5. In which region are concurrent assertions sampled and evaluated? Their signals are sampled in the Preponed region and the assertion is evaluated in the Observed region. Their action blocks run in the Reactive region.

Q6. What is a delta cycle? One pass through the Active–Inactive–NBA loop without simulation time advancing. Several delta cycles may be needed within one time step for all signals to settle.

1.5 Procedural Statements at a Glance

Functions and tasks are built from the following statements. Loops were covered with arrays in Module 2; this table is a reference.

Statement	Form	Notes
Conditional	`if (c) ... else if (d) ... else ...`	An X or Z condition is treated as false (Module 1, Section 4.3)
`unique if`, `priority if`	`unique if (a) ... else if (b) ...`	`unique`: exactly one condition true, otherwise a violation report. `priority`: at least one true
Selection	`case`, `casez`, `casex`, `case (...) inside`	`unique case` and `priority case` add the same checks

Statement	Form	Notes
Loops	`for`, `foreach`, `while`, `do ... while`, `repeat (n)`, `forever`	`forever` needs a timing control or a `break` inside
Jumps	`break`, `continue`, `return`	Section 1.2
Delay	`#10;` `#(period/2);`	Suspends the process for a time
Event control	`@(posedge clk);` `@(a or b);` `@(ev);`	Suspends until the event occurs **after** this statement is reached
Level-sensitive wait	`wait (ready == 1);`	Continues at once if the condition is already true; otherwise suspends until it becomes true
Assignment	`=`, `<=`, `+=`, `++` and the other operators	`++` and `+=` are blocking assignments
Subroutine call	`my_task(a, b);` `y = my_func(a);`	The rest of this module
Parallel block	`fork ... join`, `join_any`, `join_none`	Module 7
Immediate assertion	`assert (cond) else $error(...);`	Module 10

@ versus wait

	`@(posedge sig)` / `@(sig)`	`wait (sig)`
Sensitive to	A change (an edge)	A level (the condition being true)
If the condition already holds when reached	Still waits for the **next** change	Does not wait
Typical use	Synchronising to a clock	Waiting for a flag such as `done` or `!busy`

This distinction returns in Module 7, where missing an event that was triggered just before `@` is reached is a standard cause of hung testbenches.

Interview questions

Q1. What is the difference between `@(posedge clk)` and `wait (clk)`? `@(posedge clk)` waits for the next rising edge, even if `clk` is already 1. `wait (clk)` continues immediately when `clk` is 1 and blocks only while it is 0.

Q2. What is the difference between `while` and `do ... while`? `while` tests the condition before each iteration, so the body may execute zero times. `do ... while` tests it after each iteration, so the body executes at least once.

Q3. What is wrong with `forever begin count++; end`? There is no timing control inside, so the loop never yields. Simulation hangs at the current time.

Part 2: Functions

2.1 Why Subroutines Exist

In plain English

Suppose a testbench needs to compute the expected ALU result in the scoreboard, again in a coverage model, and again in a debug message. Writing the same `case` statement three times means three places to update when an operation is added, and three chances to make a mistake. A **subroutine** is a named piece of code written once and executed ("called") from as many places as needed, each time with different input values.

SystemVerilog has two kinds of subroutine:

	Function	Task
One-line description	Computes something and returns immediately	Performs an activity that may take simulation time
Analogy	A calculator: you get the answer at once	A job: "send this packet", which takes several clock cycles

Why they are needed

Benefit	Example
Reuse	One `alu_model()` function used by the scoreboard and by coverage
Readability	`reset_dut(); drive_op(...); check(...);` reads like the test plan
One place to change	A protocol change is made in one `drive` task
Abstraction	A test calls `write_reg(addr, data)` and does not care about the pin-level sequence
Building block of OOP	Class methods are functions and tasks (Module 4)

Parts 2 and 3 describe functions and tasks separately. Part 4 describes arguments, which work the same way for both.

2.2 Functions in SystemVerilog

In plain English

A function takes some inputs, computes a result, and gives that result back to the place where it was called. It does this **instantly**: no simulation time passes between the call and the return. Because of that, a function call can be used inside an expression just like a variable: `y = max(a, b) + 1;`.

Technical definition

A function is a subroutine that

- executes in **zero simulation time** (it may not contain `#`, `@`, `wait` or any other blocking statement),

- **returns a single value** of a declared type, or is declared `void` and returns nothing,
- can be called as an operand in an expression,
- may have any number of arguments, including none, with directions `input`, `output`, `inout` and `ref`.

Syntax

```
function [lifetime] [return_type] function_name ([arguments]);
  [local declarations]
  statements
  return expression;
endfunction [: function_name]
```

Element	Meaning	If omitted
`lifetime`	`automatic` or `static`	`static` in a module, interface, program or package; always automatic in a class (Section 5.1)
`return_type`	Any data type, or `void`	**1-bit `logic`**
arguments	`direction type name`, comma separated	Direction defaults to `input`; type defaults to `logic`

Three equivalent ways of writing the same function:

```
// 1. SystemVerilog (ANSI) style: preferred
function automatic int add(input int x, input int y);
  return x + y;
endfunction

// 2. Same, relying on defaults: direction defaults to input, y inherits "int" from x
function automatic int add(int x, y);
  return x + y;
endfunction

// 3. Verilog-1995 style: arguments declared in the body, result assigned to the function name
function integer add;
  input integer x;
  input integer y;
  begin
    add = x + y;
  end
endfunction
```

`begin`...`end` inside a function or task is optional in SystemVerilog; several statements may be written directly between `function` and `endfunction`.

How it works internally

```
   y = add(3, 4) + 1;

   1. The caller evaluates the actual arguments:        3, 4
   2. Storage for the function's variables is obtained:  x, y, and an implicit variable
        - automatic function: a fresh set for this call
        - static function   : the single permanent set shared by all calls
   3. Input arguments are copied in:                     x = 3, y = 4
   4. The body executes. No timing control is possible, so nothing else in the
      simulation runs in between.
   5. "return x + y" stores 7 as the function's result and ends the function.
   6. Output/inout arguments (if any) are copied back to the caller.
   7. The call "add(3, 4)" in the expression is replaced by 7:   y = 7 + 1 = 8
```

Inside every non-void function there is an implicit variable with the **same name as the function** and the return type. Assigning to it sets the return value. That is how Verilog functions returned values, and it still works.

Complete example

tb_function_basic.sv

```
module tb_function_basic;

  // 1. Return with the return statement
  function automatic int add(int x, int y);
    return x + y;
  endfunction

  // 2. Return by assigning to the function name (Verilog style)
  function automatic int max3(int x, int y, int z);
    max3 = x;
    if (y > max3) max3 = y;
    if (z > max3) max3 = z;
  endfunction

  // 3. Return type omitted: the function returns ONE BIT of type logic
  function automatic add_one_bit(int x, int y);
    return x + y;
  endfunction

  // 4. A function with no arguments
  function automatic string banner();
    return "=== ALU test ===";
  endfunction

  // 5. A function that never sets its return value
  function automatic int forgot_return(int x);
    int unused;
    unused = x * 2;
  endfunction

  int r;

  initial begin
    r = add(3, 4);                 $display("1. add(3, 4)            = %0d", r);
    r = add(add(1, 2), 10) * 2;    $display("2. add(add(1, 2), 10)*2 = %0d", r);
    r = max3(7, 42, 19);           $display("3. max3(7, 42, 19)      = %0d", r);
    r = add_one_bit(3, 4);         $display("4. add_one_bit(3, 4)    = %0d", r);
    $display("5. %s", banner());
    r = forgot_return(5);          $display("6. forgot_return(5)     = %0d", r);
  end

endmodule
```

EXPECTED OUTPUT

```
1. add(3, 4)            = 7
2. add(add(1, 2), 10)*2 = 26
3. max3(7, 42, 19)      = 42
4. add_one_bit(3, 4)    = 1
5. === ALU test ===
6. forgot_return(5)     = 0
```

Line by line

Line	Explanation
`function automatic int add(int x, int y);`	Returns an `int`. Two `int` inputs. `automatic` gives each call its own storage; use it by default (Section 5.1).
`return x + y;`	Sets the result and leaves the function immediately.

Line	Explanation
`max3 = x;`	Assigns to the implicit variable named after the function. The function continues executing; the value held by `max3` when `endfunction` is reached is returned.
`function automatic add_one_bit(...)`	No return type, so it is `logic` (1 bit). The sum 7 is `...0111`; only the least significant bit, 1, is returned.
Output line 2	A function call is an expression, so calls can be nested and combined with operators: (3 + 10) × 2 = 26.
`banner()`	A function without arguments. It can be called with or without the empty parentheses; writing them makes it clear that it is a call.
`forgot_return`	Neither `return` nor an assignment to the name was executed, so the implicit variable keeps its default: 0 for `int`. For a `logic` return type it would be X.

Where functions are used

Place	Example	Notes
Right-hand side of any assignment	`exp = alu_model(a, b, op);`	
Continuous assignment	`assign parity = calc_parity(data);`	Synthesizable when the function is
`always_comb`	`always_comb y = encode(x);`	`always_comb` is sensitive to signals the function reads
Conditions	`if (is_valid(pkt)) ...`	
Constraints	`constraint c { len == calc_len(kind); }`	Function is evaluated before solving (Module 5)
Class methods	`txn.print(); t2 = t1.copy();`	Module 4
Parameters and constants	`localparam int W = clog2_fn(DEPTH);`	Only "constant functions": automatic or static, no side effects, all inputs constant

Synthesis

A function is synthesizable when it contains only synthesizable statements, which is naturally the case for most: no delays are possible anyway. It becomes a block of combinational logic, duplicated at every place it is called. Functions should be declared `automatic` in RTL so that simulation matches the synthesised logic.

> **Interview trap:** "What does a function return if you leave out the return type?" One bit of `logic`. A function `function sum(int a, int b)` silently truncates its result to the LSB.

> **Common mistake:** Using both styles in one function. After `return expr;` the function ends, so later assignments to the function name are never reached. And a value assigned to the function name is discarded if a `return other_value;` follows. Choose one style: `return`.

> **Common mistake:** Assuming `return` is needed at the end of a function that assigns to its own name. It is not; reaching `endfunction` returns the current value of the implicit variable.

> **Verification application:** The scoreboard's reference model is a function: `function logic [8:0] alu_model(bit [7:0] a, b, alu_op_e op);`. It has no timing, depends only on its inputs, and is called once per observed transaction. Keeping the model in one function means that the scoreboard, a coverage sampler and a debug printout all agree on the expected value.

Interview questions

Q1. What is a function? A subroutine that executes in zero simulation time, cannot contain timing controls, and returns a single value (or is `void`). It can be used inside expressions.

Q2. What is the default return type of a function? A 1-bit `logic`.

Q3. How can a function return its value? With `return expression;`, or by assigning to the implicit variable that has the function's name. If neither happens, the default value of the return type is returned.

Q4. Can a function have `output` arguments? Yes. SystemVerilog functions can have `input`, `output`, `inout` and `ref` arguments. (Verilog functions allowed only inputs.) Such a function cannot be called where side effects are not allowed: in a continuous assignment or any other expression outside procedural code, or in an event expression.

Q5. Is a function synthesizable? Yes, provided its body is synthesizable. It is implemented as combinational logic.

Q6. Can a function be called without using its return value? The standard expects the result to be used. To discard it deliberately, cast the call to void: `void'(my_func(x));`. Without the cast, simulators issue a warning.

2.3 Returning Values and Void Functions

Returning different kinds of value

A function returns exactly one value, but that value can be of any data type, including aggregates:

Return type	Declaration	Notes
Scalar or vector	`function logic [8:0] alu_model(...);`	
Enum, packed struct	`function alu_op_e next_op(alu_op_e op);`	Any user-defined type name
Unpacked struct	`function item_t make_item(...);`	Through a `typedef` name
Unpacked array, dynamic array, queue	`function buf_t make_ramp(...);`	The array type must be a `typedef` (Module 1, Section 3.6)
`string`	`function string to_str();`	Very common: `convert2string` in UVM
`real`	`function real average(int q[$]);`	Simulation-only
Class handle	`function alu_txn copy();`	Returns a handle to an object (Module 4)
Nothing	`function void print();`	Void function

To return **more than one** value there are three options: return a struct, add `output` arguments, or add `ref` arguments. A common pattern is to return a status bit and deliver the data through an `output` argument.

Void functions

In plain English. Sometimes a subroutine does not compute anything to give back; it just does something, such as printing a message or updating a counter, and it does so instantly. That is a **void function**.

Technical definition. A function declared with return type `void` returns no value. It is called as a statement, not inside an expression. It keeps every restriction of a function: zero time, no blocking statements.

Why not use a task for that? A task is allowed to consume time, so the compiler must assume that it might. That has two consequences. First, a function cannot call a task, so a helper written as a task cannot be used from any function, from `always_comb`, or from a constraint. Second, a reader cannot tell from the call whether a task blocks. Declaring the helper as `void function` guarantees, and documents, that it returns immediately, and lets it be called from anywhere.

> **Key point:** Rule used across the industry and in UVM: **if a subroutine does not need to consume time, make it a function** (`void` if it has nothing to return). Use a task only when the subroutine must wait for something.

Discarding a return value with void'()

```
void'(txn.randomize());          // legal, but a bad habit: the result reports failure
void'(mbx.try_get(txn));         // "I do not care whether an item was available"
```

A non-void function call written as a bare statement draws a warning; the `void'()` cast states that discarding the result is intentional. Be careful with what you discard. Throwing away the result of `randomize()` hides randomization failures (Module 5).

Complete example

tb_function_return.sv

```
module tb_function_return;

  typedef struct {
    int quotient;
    int remainder;
  } divmod_t;

  int unsigned error_count;

  // Void function: acts, returns nothing
  function automatic void report(string msg, bit is_error = 0);
    if (is_error) begin
      error_count++;
      $display("ERROR: %s", msg);
    end
    else $display("INFO : %s", msg);
  endfunction

  // Two results, method 1: return a struct
  function automatic divmod_t divmod(int x, int y);
    divmod_t r;
    r.quotient  = x / y;
    r.remainder = x % y;
    return r;
  endfunction

  // Two results, method 2: status as the return value, data through an output argument
  function automatic bit safe_div(input int x, input int y, output int q);
    if (y == 0) return 0;      // q keeps its default value
    q = x / y;
```

```
    return 1;
  endfunction

  // A void function may use "return;" to leave early
  function automatic void print_if_odd(int x);
    if (x % 2 == 0) return;
    $display("%0d is odd", x);
  endfunction

  divmod_t dm;
  int      q;

  initial begin
    report("starting");
    dm = divmod(17, 5);
    $display("17 / 5 = %0d remainder %0d", dm.quotient, dm.remainder);

    if (safe_div(20, 4, q))  $display("20 / 4 = %0d", q);
    if (!safe_div(20, 0, q)) report("division by zero rejected", 1);

    print_if_odd(4);
    print_if_odd(7);

    void'(safe_div(9, 3, q));
    $display("q = %0d, errors = %0d", q, error_count);
  end

endmodule
```

EXPECTED OUTPUT

```
INFO : starting
17 / 5 = 3 remainder 2
20 / 4 = 5
ERROR: division by zero rejected
7 is odd
q = 3, errors = 1
```

Line by line

Line	Explanation
`function automatic void report(string msg, bit is_error = 0);`	A void function with a default argument (Section 4.4). It modifies the module variable `error_count`: a function may read and write variables outside itself.
`report("starting");`	Called as a statement. No `void'()` is needed because there is no return value.
`function automatic divmod_t divmod(...)`	The return type is a struct, so two numbers come back in one value.
`function automatic bit safe_div(input int x, input int y, output int q);`	Returns 1 for success. The quotient is delivered through the `output` argument.
`if (y == 0) return 0;`	Early return. The output `q` was never assigned, so the caller receives the default value of the formal argument, 0 (Section 4.1).
`if (x % 2 == 0) return;`	In a void function `return;` has no expression.
`void'(safe_div(9, 3, q));`	The status is deliberately ignored; `q` is still written.

Function versus void function

Property	Function with a return type	Void function
Returns a value	Yes	No

Property	Function with a return type	Void function
Used in an expression	Yes	No, only as a statement
Called as a statement	Only with `void'()` (otherwise a warning)	Yes
`return` form	`return expr;`	`return;`
May consume time	No	No
Typical use	Calculations, queries, `copy()`, `compare()`	`print()`, `report()`, `connect()`, setters

Interview questions

Q1. What is a void function? A function that returns no value. It is called as a statement and, like every function, executes in zero simulation time.

Q2. When would you write a void function instead of a task? Whenever the subroutine does not need to consume time. A void function can be called from other functions and from contexts where tasks are illegal, and it guarantees to the reader that the call does not block.

Q3. How do you return more than one value from a function? Return a struct (or an array or a class handle), or use `output` or `ref` arguments in addition to the return value.

Q4. What does `void'(f())` mean? It calls the function `f` and explicitly discards its return value, which suppresses the warning for an unused function result.

Q5. Can a function return an array? Yes, if the array type has a `typedef` name that is used as the return type.

2.4 Why a Function Cannot Consume Simulation Time

The rule

A function must begin and end in the same simulation time step, and without suspending. Therefore the following are **illegal inside a function**:

Illegal in a function	Why
`#delay`	Would advance time
`@(event)`	Would suspend until the event
`wait (condition)`	Might suspend
`fork` ... `join`, `fork` ... `join_any`	The parent would wait for child processes
A call to a task	The task **might** consume time, even if this particular one does not
Blocking calls such as `mailbox.get()`, `semaphore.get()`	They are tasks that can suspend
`wait_order`, `expect`, `##` cycle delay	Blocking statements

Legal in a function: blocking assignments, `if`, `case`, loops, calls to other functions (including itself), system functions and non-blocking system tasks such as `$display`, non-blocking methods such as `mailbox.try_get()`, and `fork` ... `join_none` (see below).

The reason

A function call is an **operand of an expression**, and an expression must be evaluated atomically.

```
assign y = a + f(b);              continuous assignment: must settle "now"

if (f(x) > g(y)) ...              the two operands must describe the same instant

always_comb z = f(a) & c;         combinational logic has no notion of waiting

constraint c { len == f(kind); }  the solver calls f() while solving; it cannot pause
```

Consider `y = a + f(b);`. The simulator reads `a`, then calls `f`. If `f` could wait for 10 ns, what would the expression mean? `a` was read at one time and `f(b)` finished at another; `a` and `b` might have changed in between; and in a continuous assignment or an `always_comb` block there is no process that *could* wait. An expression has a value at an instant. The language keeps that true by requiring every function to complete in zero time. Anything that must wait is a different kind of thing, and SystemVerilog gives it a different keyword: `task`.

The same guarantee is what makes functions synthesizable as pure combinational logic, and what allows a simulator to call them from regions and contexts where suspension is impossible (continuous assignments, constraint solving, `final` blocks, constructors, coverage sampling).

Why even a task with no delays is forbidden

```
task automatic no_delay_task();
  $display("no timing control in here");
endtask

function automatic void f();
  no_delay_task();       // ILLEGAL: a function cannot call a task
endfunction
```

The check is made on the **kind** of subroutine, not on its contents. Legality must be decidable at compile time from the declaration alone, and must not change when someone later edits the task body. With virtual methods (Module 4) the compiler cannot even know which task body will run. So the rule is absolute: no task calls in a function.

The one exception: fork...join_none

A function may contain `fork`...`join_none`. The function does not wait: it spawns background processes and returns immediately, still in zero time. The spawned processes begin executing after the calling process next suspends, and **they** are free to use delays and call tasks.

tb_function_no_time.sv

```
module tb_function_no_time;

  // Legal: the function itself returns immediately; the forked thread runs later
  function automatic void start_watchdog(int timeout);
    fork
      begin
        #timeout;
        $display("[%0t] watchdog expired", $time);
      end
    join_none
    $display("[%0t] start_watchdog() returns", $time);
  endfunction

  // function automatic int bad(int x);
  //   #5;                      // ILLEGAL: delay in a function
  //   @(posedge clk);          // ILLEGAL: event control in a function
  //   return x;
  // endfunction

  initial begin
    $display("[%0t] calling function", $time);
    start_watchdog(20);
    $display("[%0t] back in the caller", $time);
    #30;
    $display("[%0t] done", $time);
  end

endmodule
```

EXPECTED OUTPUT

```
[0] calling function
[0] start_watchdog() returns
[0] back in the caller
[20] watchdog expired
[30] done
```

Line by line

Line	Explanation
`fork ... join_none`	Creates a child process containing the `begin`...`end` block and continues **without waiting**.
`#timeout;` inside the fork	Legal: it is executed by the child process, not by the function.
First three output lines	All at time 0. The function printed and returned without any time passing.
`[20] watchdog expired`	The child process ran in the background after the `initial` block suspended at `#30`.
Commented `bad` function	Uncomment it to see the compile error your simulator gives for a timing control in a function.

This pattern is used in real testbenches: a class constructor (`new` is a function) or a UVM function phase that needs to start a background monitor thread does it with `fork`...`join_none`. The standard requires the function to be called from procedural code that originates in an `initial` or `always`-type process, not from a continuous assignment.

Nonblocking assignments and event triggers in functions

Recent revisions of IEEE 1800 allow a function to contain nonblocking assignments (`<=`) and event triggers (`->`), since neither suspends the function. Verilog-2001 did not allow nonblocking assignments in functions, some tools still restrict them, and such a function cannot be used where side effects are forbidden. For synthesizable code, functions should contain only blocking assignments to their own local variables and return value.

What the question is really testing

Question	Short answer
Can a function have a delay?	No
Can a function call a task?	No (except from inside a `fork` ... `join_none` within it)
Can a task call a function?	Yes
Can a function call another function?	Yes, including itself (recursion)
Can a function contain `fork` ... `join`?	No. Only `fork` ... `join_none`
Can a function call `$display`?	Yes. It is a system task that does not block
Can a function call `mailbox.get()`?	No, `get()` is a blocking task. `try_get()` is a function and is allowed
Can a constructor `new()` contain a delay?	No. `new` is a function
Is `function void` allowed to consume time, since it returns nothing?	No. `void` only removes the return value

> **Interview trap:** "The task contains no delay, so can my function call it?" No. The restriction is on calling a task at all, regardless of what is inside it. Convert the task into a void function.

> **How to say it in an interview:** "A function is used as an operand in expressions, in continuous assignments, in `always_comb` and in constraints. An expression must be evaluated in zero time, so a function is not allowed to contain any statement that could suspend it: no delays, no event controls, no waits and no task calls. A task has no return value and is called as a statement, so it is allowed to block. The only timing-related construct permitted in a function is `fork` ... `join_none`, because the function still returns immediately."

Interview questions

Q1. Why can a function not contain a delay? Because a function call is part of an expression and must produce its value in the same time step in which it is called. A delay would suspend the evaluation of the expression, which is meaningless in a continuous assignment, combinational block or constraint.

Q2. Can a function call a task? Can a task call a function? A function cannot call a task, because the task might consume time. A task can call any function.

Q3. Is there any way for a function to start time-consuming activity? Yes: with `fork` ... `join_none`. The function spawns a background process and returns immediately; the spawned process may contain delays and task calls.

Q4. Can you use `wait` or `@` in a function if the condition is already true? No. The statement is illegal in a function whether or not it would actually block.

Q5. Why is a constructor not allowed to wait for a clock edge? `new()` is a function. Object construction happens in zero time. Anything that must wait is placed in a task of the class, typically `run()`, which is called after construction.

Part 3: Tasks

3.1 Tasks in SystemVerilog

In plain English

A task is a named sequence of actions that is allowed to take time. "Apply reset for two clock cycles", "send one byte on the bus", "wait until the DUT says it is ready": each of these lasts for a while in simulation time, and each is something you want to write once and call many times. That is what a task is for.

A task does not hand back a value the way a function does. If it has results, it delivers them through `output`, `inout` or `ref` arguments.

Technical definition

A task is a subroutine that

- **may** contain timing controls (`#`, `@`, `wait`) and other blocking statements, and may therefore consume simulation time (it does not have to),
- has **no return value**; it is called as a statement and cannot be used in an expression,
- may have any number of `input`, `output`, `inout` and `ref` arguments, including none,
- may call other tasks and functions.

Syntax

```
task [lifetime] task_name ([arguments]);
  [local declarations]
  statements
endtask [: task_name]
```

```
// SystemVerilog (ANSI) style
task automatic wait_cycles(input int n);
  repeat (n) @(posedge clk);
endtask

// Verilog-1995 style: arguments declared in the body
task wait_cycles_v;
  input integer n;
  begin
    repeat (n) @(posedge clk);
  end
endtask
```

A task with no arguments may be called with or without parentheses: `reset_dut();` or `reset_dut;`.

How it works internally

```
initial begin
  stmt_A;
  send_byte(8'hA5);     <-- call
  stmt_B;
end

1. Actual arguments are evaluated; input and inout values are copied into the formals.
2. The CALLING PROCESS itself starts executing the task body. No new process is
   created: a task call is not a fork.
3. When the body reaches @(posedge clk) or #10, the calling process suspends there.
   Other processes run; simulation time advances.
4. When the wait is over, the process resumes inside the task.
5. At endtask (or "return;"), output and inout formals are copied back to the
   caller's variables.
6. The calling process continues with stmt_B.

Consequence: stmt_B does not execute until the whole task has finished.
A task call BLOCKS its caller for as long as the task takes.
```

Two properties of this mechanism cause most task-related bugs, and both are examined later:

- Output arguments are copied back **only when the task returns** (Section 4.1).
- In a `static` task, the formals and local variables are a single shared copy, so two processes inside the same task at the same time overwrite each other (Section 5.2).

Complete example

tb_task_basic.sv

```
module tb_task_basic;

  logic       clk = 0;
  logic       valid = 0;
  logic [7:0] data;

  always #5 clk = ~clk;

  // A task with an input argument and timing control
  task automatic wait_cycles(input int n);
    repeat (n) @(posedge clk);
  endtask

  // A task that drives module-level signals over two clock cycles
  task automatic send_byte(input logic [7:0] value);
    @(posedge clk);
    valid <= 1'b1;
    data  <= value;
    @(posedge clk);
    valid <= 1'b0;
  endtask

  // A task with no timing control: legal, but it should have been a void function
  task automatic show(input string msg);
    $display("[%0t] %s", $time, msg);
  endtask

  // A simple monitor
  always @(posedge clk)
    if (valid) $display("[%0t] monitor saw data = %h", $time, data);

  initial begin
    show("start");
    wait_cycles(2);
    show("after wait_cycles(2)");
    send_byte(8'hA5);
    #1;
    show("after send_byte");
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[0] start
[15] after wait_cycles(2)
[35] monitor saw data = a5
[36] after send_byte
```

Line by line

Line	Explanation
`task automatic wait_cycles(input int n);`	One input. The body waits for `n` rising edges, so the task consumes `n` clock cycles.
`wait_cycles(2);`	Called at time 0. Rising edges occur at 5 and 15, so the call returns at 15. The caller was blocked for 15 time units.
`send_byte(8'hA5);`	Called at 15, just after that edge. The first `@(posedge clk)` waits for the **next** edge, at 25.

Line	Explanation
`valid <= 1'b1; data <= value;`	Nonblocking drives at 25. The monitor block, which runs at the same edge, still sees `valid` = 0. This is how a real registered output behaves, and it is why testbench drives use `<=`.
Second `@(posedge clk)`	At 35 the monitor samples `valid` = 1 and `data` = `A5` and prints. The task then schedules `valid <= 0` and returns.
`#1;`	Moves the final message away from the clock edge, so that its order relative to the monitor's message is not left to the simulator.
`show(...)`	A task without any timing control. It works, but nothing can call it from a function. A void function is the better choice.

A task can read and write any variable that is visible where the task is declared (`clk`, `valid`, `data` here). This is convenient in a small module-level testbench. In a class-based testbench the same signals are reached through a virtual interface (Module 8).

Where tasks are used

Use	Example
Pin-level protocol sequences	`drive(txn)`, `write_reg(addr, data)`, `read_reg(addr, data)`
Reset and initialisation sequences	`reset_dut()`
Waiting for conditions	`wait_for_ready()`, `wait_cycles(n)`
The main loop of every testbench component	`task run(); forever begin ... end endtask` in the driver, monitor, scoreboard
Bus functional models (BFMs)	A module or interface containing tasks that perform bus transfers

Synthesis

A task is synthesizable only if it contains no timing control, which makes it equivalent to a void function. In practice tasks are a verification construct; RTL uses functions.

> **Common mistake:** Trying to use a task in an expression: `x = my_task(a);` is a compile error. A task has no return value. Use an `output` argument, or make it a function if it does not consume time.

> **Common mistake:** Thinking that calling a task starts it "in the background". The caller waits until the task returns. To run a task in the background, call it inside `fork` ... `join_none` (Module 7).

> **Verification application:** The driver's job in the ALU testbench is a task: wait for a clock edge, put `a`, `b`, `op` on the interface with `valid_in` = 1, wait one cycle, remove `valid_in`. It must be a task because it spans clock cycles. The transaction's `print()` is a function because it finishes instantly.

Interview questions

Q1. What is a task? A subroutine that can contain timing controls and so can consume simulation time. It has no return value, is called as a statement, and passes results back through `output`, `inout` or `ref` arguments.

Q2. Must a task contain a delay? No. A task *may* consume time; it is not required to. But a subroutine that never needs to wait should be written as a function.

Q3. Does calling a task create a new thread? No. The task body is executed by the calling process, which is blocked until the task returns. A new thread is created only by `fork`.

Q4. Can a task return a value? Not with `return value;`. A task can use `return;` (with no expression) to exit early, and delivers values through its `output`, `inout` or `ref` arguments.

Q5. Is a task synthesizable? Only when it contains no timing controls. Tasks are normally used in testbenches.

3.2 Timing Control Inside Tasks

The three timing controls

Control	Syntax	The task resumes when
Delay	`#10;` `#(period);` `#1ns;`	The given amount of simulation time has passed
Event control	`@(posedge clk);` `@(negedge rst_n);` `@(sig);` `@(ev);`	The next occurrence of the edge, change or named event
Level-sensitive wait	`wait (ready);` `wait (count == 0);`	The condition is true (immediately, if it already is)

A task may also block in other ways: calling another task that blocks, `mailbox.get()`, `semaphore.get()`, `fork`...`join`, `wait fork` (Module 7).

Which one to use

Situation	Use	Why
Synchronous DUT interface	`@(posedge clk)` or a clocking block event `@(vif.drv_cb)`	The testbench stays aligned with the clock whatever the period is
Clock and reset generation, asynchronous stimulus	`#delay`	These define time itself
Waiting for a status flag	`wait (flag)`, or a loop of `@(posedge clk)` that tests the flag	`wait` alone is sensitive to glitches; sampling on the clock is safer for DUT signals
Protection against hanging	A timeout in parallel with the wait	Module 7 (`fork`...`join_any` with `disable fork`)

> **Key point:** In a testbench for a clocked design, tasks should advance time with clock edges, not with `#` delays. A task written as `#10; valid = 1; #10; valid = 0;` breaks as soon as the clock period changes and can place stimulus exactly on the clock edge, creating a race.

Execution trace of a task with timing

tb_task_timing.sv

```
module tb_task_timing;

  logic clk = 0;
  logic rst_n;
  logic ready = 0;

  always #5 clk = ~clk;

  initial #42 ready = 1;             // "DUT" raises ready at time 42

  task automatic reset_dut(input int cycles);
    rst_n = 1'b0;
    repeat (cycles) @(posedge clk);
    @(negedge clk);
    rst_n = 1'b1;
    $display("[%0t] reset released", $time);
  endtask

  task automatic wait_ready();
    $display("[%0t] waiting for ready", $time);
    wait (ready);
    $display("[%0t] ready is high", $time);
    @(posedge clk);
    $display("[%0t] aligned to the clock", $time);
  endtask

  initial begin
    reset_dut(2);
    wait_ready();
    wait_ready();
    #3;
    $display("[%0t] end", $time);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[20] reset released
[20] waiting for ready
[42] ready is high
[45] aligned to the clock
[45] waiting for ready
[45] ready is high
[55] aligned to the clock
[58] end
```

Execution flow

Time	What happens
0	`reset_dut(2)` drives `rst_n` low and waits.
5, 15	The two rising edges counted by `repeat (2)`.
20	`@(negedge clk)`: reset is released on a falling edge, away from the active edge. The task returns.
20	First `wait_ready()`: `ready` is 0, so `wait (ready)` suspends.
42	`ready` becomes 1. The `wait` ends at once, in the middle of a clock cycle.
45	`@(posedge clk)` brings the task back in step with the clock. It returns.
45	Second `wait_ready()`: `ready` is already 1, so `wait (ready)` does not block at all.

Time	What happens
55	`@(posedge clk)` waits for the next edge, even though the task was called exactly at an edge. The edge at 45 has already happened.
58	`#3` after the return.

This example shows the difference between a level-sensitive `wait` (passes straight through when the condition is true) and an edge-sensitive `@` (always waits for a future event).

Tasks and the clock edge

```
   clk      __|‾‾|__|‾‾|__|‾‾|__|‾‾|__
   time       5    15    25    35

   task automatic drive_op(...);
     @(posedge clk);          <- the caller is suspended until the edge
     valid_in <= 1'b1;        <- nonblocking: the DUT at THIS edge still sees the old value,
     a        <= a_i;            and sees the new values at the NEXT edge. No race.
     @(posedge clk);
     valid_in <= 1'b0;
   endtask
```

Driving with blocking assignments exactly at the active edge creates a race: whether the DUT samples the old or the new value depends on which process the simulator runs first. The three standard cures are nonblocking assignments (above), driving on the opposite edge (as the Module 1 testbench did), or a clocking block (Module 8).

Leaving a task early

```systemverilog
task automatic wait_for_done(input int max_cycles);
  repeat (max_cycles) begin
    @(posedge clk);
    if (done) return;                 // leave the task as soon as done is seen
  end
  $error("done was not seen within %0d cycles", max_cycles);
endtask
```

`return;` ends the task at that point; output arguments are copied back as usual. `disable task_name;` also ends a task, but it kills **every** process currently executing that task, and output values are not guaranteed to be copied; it is a blunt Verilog-era tool and `return` is preferred.

Common mistake: Calling a task that waits for an event that has already happened. `@(posedge start)` reached after `start` has already risen waits for the next rising edge, possibly for ever. Use `wait (start)` when "already true" should count.

Verification application: Every component of the layered testbench has a `task run()` built from these controls. The driver's `run` is `forever begin gen2drv_mbx.get(txn); drive(txn); end`: it blocks on the mailbox until a transaction arrives, then blocks for the clock cycles needed to drive it. The whole behaviour of the testbench in time is the sum of the blocking points in its tasks.

Interview questions

Q1. Which timing controls can a task contain? Delays (`#`), event controls (`@`) and level-sensitive waits (`wait`), as well as any other blocking statement such as a blocking task call, `mailbox.get()` or `fork` ... `join`.

Q2. What is the difference between `#10` and `@(posedge clk)` inside a driver task? `#10` waits a fixed time regardless of the clock, so the stimulus drifts or races if the clock period or phase differs. `@(posedge clk)` waits for the actual clock edge, so the task stays synchronous with the DUT.

Q3. A task is called at the exact time of a rising clock edge and begins with `@(posedge clk)`. When does it continue? At the next rising edge, one full period later. The edge at the time of the call has already occurred.

Q4. How do you exit a task before its end? With `return;`. The older alternative is `disable` on the task or on a named block inside it.

3.3 Differences Between Functions and Tasks

Comparison table

Property	Function	Task
Simulation time	Always zero	Zero or more
Timing controls (`#`, `@`, `wait`)	Not allowed	Allowed
Return value	One value, or `void`	None
Use in an expression	Yes (non-void)	No
Called as a statement	Void functions; others with `void'()`	Yes
Arguments	`input`, `output`, `inout`, `ref`; zero or more	`input`, `output`, `inout`, `ref`; zero or more
Can call functions	Yes	Yes
Can call tasks	No (only inside `fork` ... `join_none`)	Yes
`fork` ... `join`, `join_any`	Not allowed	Allowed
`fork` ... `join_none`	Allowed	Allowed
`return` statement	`return value;` (`return;` if void)	`return;` only
Usable in `assign`, `always_comb`, constraints	Yes	No
Synthesizable	Yes	Only without timing controls
Default lifetime in a module	Static	Static
Default lifetime in a class	Automatic	Automatic
Typical testbench use	`print`, `copy`, `compare`, reference model, `new`	`run`, `drive`, `monitor`, `reset`, anything that waits

In old Verilog two more differences existed: a function needed at least one input and could not have outputs. SystemVerilog removed both restrictions, so do not quote them as SystemVerilog rules.

The same job written both ways

tb_func_vs_task.sv

```
module tb_func_vs_task;

  logic clk = 0;
  always #5 clk = ~clk;

  // Function: answer available immediately, usable inside an expression
  function automatic logic [8:0] add9(logic [7:0] x, logic [7:0] y);
    return {1'b0, x} + {1'b0, y};
  endfunction

  // Task: same calculation, but the answer is delivered one clock cycle later,
  // the way the real ALU delivers it
  task automatic add9_clocked(input  logic [7:0] x,
                              input  logic [7:0] y,
                              output logic [8:0] sum);
    @(posedge clk);
    sum = {1'b0, x} + {1'b0, y};
  endtask

  logic [8:0] r;

  initial begin
    r = add9(8'd200, 8'd100);
    $display("[%0t] function result = %0d", $time, r);

    if (add9(8'd255, 8'd1) == 9'd256)
      $display("[%0t] a function call can sit inside a condition", $time);

    add9_clocked(8'd200, 8'd100, r);
    $display("[%0t] task result     = %0d", $time, r);
    $finish;
  end

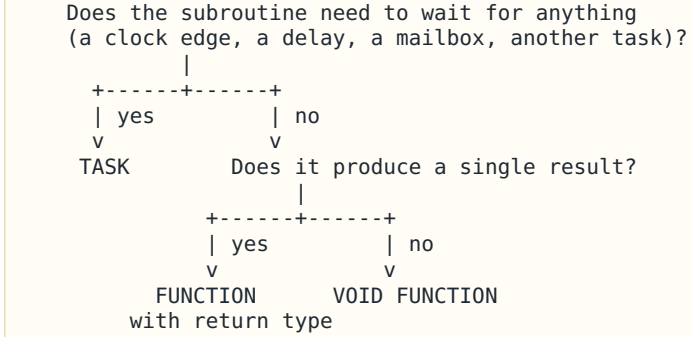
endmodule
```

EXPECTED OUTPUT

```
[0] function result = 300
[0] a function call can sit inside a condition
[5] task result     = 300
```

The function gives its answer at the time of the call and can be embedded in an `if`. The task must be called as a separate statement, blocks the caller until the clock edge at time 5, and passes its answer back through the `output` argument.

How to choose



> **Interview trap:** "A task can do everything a function can and more, so why have functions?" Because a function carries a guarantee: it returns in zero time. That guarantee is what allows it to be used in expressions, continuous assignments, `always_comb`, constraints, constructors and `final` blocks. A task gives no such guarantee and is excluded from all of them.

> **Common mistake:** Saying "a function returns a value and a task does not" as the only difference. The primary difference is **time**. Return value comes second.

Interview questions

Q1. What are the differences between a function and a task? (1) A function executes in zero time and cannot contain timing controls; a task can consume time. (2) A function returns a value (unless void) and can be used in an expression; a task returns nothing and is called as a statement. (3) A function cannot call a task; a task can call both. (4) A function is synthesizable; a task only if it has no timing. Both accept `input`, `output`, `inout` and `ref` arguments.

Q2. Can a function have output arguments like a task? Yes, in SystemVerilog.

Q3. When would you choose a void function over a task? When the subroutine returns nothing but also never waits. It can then be called from functions and other zero-time contexts, and readers know it does not block.

Q4. Which is used for the driver's main loop and which for a transaction's compare method? Why? The driver's loop is a task because it waits for clock edges and for mailbox items. `compare` is a function because it only inspects data and returns a pass/fail bit.

Q5. Can a task be called from a continuous assignment or `always_comb`? No from a continuous assignment. `always_comb` does not allow blocking statements, so a task that consumes time is illegal there; use functions.

3.4 Tasks Calling Other Tasks and Functions

The calling rules

Caller ↓ / Callee →	Function	Task
Function	Allowed	**Not allowed** (only inside a `fork` ... `join_none`)
Task	Allowed	Allowed
`initial`, `always`, `always_ff` body	Allowed	Allowed (`always_ff` and `always_comb` only if it does not block)
`always_comb`, `assign`, constraint	Allowed	Not allowed
`final` block	Allowed	Not allowed if it could consume time

Why layering subroutines matters

Real testbench code is layered. A test calls a high-level task; that task calls protocol-level tasks; those call signal-level tasks and helper functions. Each layer hides detail from the one above.

```
test:       run_smoke_test()                          (task)
                 |
                 +--> reset_dut()                     (task)   uses wait_cycles()
                 |
                 +--> do_op(a, b, op)                 (task)
                          |
                          +--> drive_op(...)          (task)   clock-level pin activity
                          +--> alu_model(a, b, op)    (function) expected value
                          +--> check(actual, expected) (function) compare and count
```

Complete example: a task-based ALU testbench

This testbench uses the `alu_pkg` package and `alu` module from Module 1 (compile `alu_pkg.sv` and `alu.sv` first). It is the Module 1 directed test rewritten with tasks and functions, and it is the last step before the class-based testbench: in Module 9 each of these subroutines becomes a method of a class.

tb_alu_tasks.sv

```
module tb_alu_tasks;
  import alu_pkg::*;

  logic       clk = 0;
  logic       rst_n;
  logic       valid_in;
  logic [7:0] a, b;
  alu_op_e    op;
  logic       valid_out;
  logic [8:0] result;

  int unsigned pass_cnt, fail_cnt;

  alu dut (.*);

  always #5 clk = ~clk;

  // ---------- low-level tasks ----------
  task automatic wait_cycles(input int n);
    repeat (n) @(posedge clk);
  endtask

  task automatic reset_dut();
    rst_n    = 1'b0;
    valid_in = 1'b0;
    a        = '0;
    b        = '0;
    op       = ALU_ADD;
    wait_cycles(2);                 // a task calling a task
    @(negedge clk);
    rst_n = 1'b1;
  endtask

  task automatic drive_op(input  logic [7:0] a_i,
                          input  logic [7:0] b_i,
                          input  alu_op_e    op_i,
                          output logic [8:0] res);
    @(negedge clk);
    a        = a_i;
    b        = b_i;
    op       = op_i;
    valid_in = 1'b1;
    @(negedge clk);
    valid_in = 1'b0;
    res      = result;
  endtask

  // ---------- functions ----------
  function automatic logic [8:0] alu_model(logic [7:0] a_i,
                                           logic [7:0] b_i,
                                           alu_op_e    op_i);
```

```
    case (op_i)
      ALU_ADD: return {1'b0, a_i} + {1'b0, b_i};
      ALU_SUB: return {1'b0, a_i} - {1'b0, b_i};
      ALU_AND: return {1'b0, a_i & b_i};
      ALU_OR : return {1'b0, a_i | b_i};
      ALU_XOR: return {1'b0, a_i ^ b_i};
      ALU_NOT: return {1'b0, ~a_i};
      ALU_SHL: return {a_i, 1'b0};
      ALU_SHR: return {2'b00, a_i[7:1]};
      default: return 'x;
    endcase
  endfunction

  function automatic void check(alu_op_e op_i, logic [8:0] actual, logic [8:0] expected);
    if (actual !== expected) begin
      fail_cnt++;
      $display("[%0t] FAIL %s: actual=%0d expected=%0d",
               $time, op_i.name(), actual, expected);
    end
    else begin
      pass_cnt++;
      $display("[%0t] PASS %s: result=%0d", $time, op_i.name(), actual);
    end
  endfunction

  // ---------- high-level task ----------
  task automatic do_op(input logic [7:0] a_i, input logic [7:0] b_i, input alu_op_e op_i);
    logic [8:0] actual;
    drive_op(a_i, b_i, op_i, actual);                  // task calling a task
    check(op_i, actual, alu_model(a_i, b_i, op_i));    // task calling functions
  endtask

  // ---------- the test ----------
  initial begin
    reset_dut();
    do_op(8'd200, 8'd100, ALU_ADD);
    do_op(8'd5,   8'd10,  ALU_SUB);
    do_op(8'hF0,  8'h3C,  ALU_AND);
    do_op(8'h81,  8'h00,  ALU_SHL);
    do_op(8'hAA,  8'h00,  ALU_NOT);
    $display("[%0t] %0d passed, %0d failed", $time, pass_cnt, fail_cnt);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[40] PASS ALU_ADD: result=300
[60] PASS ALU_SUB: result=507
[80] PASS ALU_AND: result=48
[100] PASS ALU_SHL: result=258
[120] PASS ALU_NOT: result=85
[120] 5 passed, 0 failed
```

Execution flow

Time	What happens
0	`reset_dut` drives reset and idle inputs, then calls `wait_cycles(2)`.
5, 15	The two rising edges. `wait_cycles` returns into `reset_dut`.
20	Falling edge: reset released. `reset_dut` returns.
20	`do_op` calls `drive_op`, which waits for the next falling edge.
30	`drive_op` applies `a`=200, `b`=100, `op`=`ALU_ADD`, `valid_in`=1.
35	Rising edge: the ALU registers the result 300.

Time	What happens
40	Falling edge: `drive_op` removes `valid_in`, copies `result` into `res` and returns; the output argument is copied to `actual`. `do_op` calls `alu_model` and `check`, both in zero time.
50, 60	The same sequence for the subtraction: drive at 50, check at 60. Each operation takes 20 time units.
120	Last check, then the summary.

Points to notice

Code	Explanation
`wait_cycles(2);` inside `reset_dut`	A task may call a task. The caller of `reset_dut` is blocked for the total time.
`output logic [8:0] res`	The only way for the task to give the sampled result back.
`alu_model`	A function: the expected value is pure calculation. `{1'b0, a_i} - {1'b0, b_i}` is evaluated in 9 bits, so 5 − 10 gives `1FB` = 507, the same wrapped value as the DUT (Module 1, Section 4.4).
`check`	A void function that updates the module-level counters. It uses `!==` so that an X result fails.
`check(op_i, actual, alu_model(a_i, b_i, op_i));`	A function call used as an argument of another function call.
`a_i`, `b_i`, `op_i`	The formal arguments have different names from the module signals `a`, `b`, `op`. If they had the same names, the argument would hide the signal inside the task, and `a = a;` would do nothing useful.
`do_op`	The test is now five readable lines. Adding a test case is one line.

Calls between parallel processes

If two processes call the same task at the same time (for example two `fork` branches both calling `drive_op`), both are executing the same code. Whether they interfere depends on the task's lifetime: with `automatic` each call has its own arguments and locals; with `static` they share them. They would still both be driving the same module signals, which needs a semaphore to arbitrate (Module 7). Section 5.2 demonstrates the lifetime problem.

A function "calling" a task through fork...join_none

```
function automatic void kick_off();
  fork
    reset_dut();          // legal here: executed by the spawned process, not by the function
  join_none
endfunction
```

As explained in Section 2.4, this is the only form in which a task call may appear inside a function body.

> **Common mistake:** Naming a formal argument the same as a module-level signal that the task is meant to drive. Inside the task the name refers to the argument. Use a suffix such as `_i`, or reach the signal through an interface handle (`vif.a`).

> **Verification application:** Compare the structure above with the layered testbench of Module 9. `drive_op` becomes `alu_driver::drive()`. Sampling `result` becomes `alu_monitor::run()`. `alu_model` and `check` become methods of `alu_scoreboard`. The five `do_op` lines become a generator producing random `alu_txn` objects. The functions-versus-tasks decision is the same in both.

Interview questions

Q1. Can a task call another task? Can it call a function? Yes to both. The calling task is blocked until the called task returns.

Q2. Can a function call a task? No, because the task might consume time. The only exception is a call placed inside a `fork`...`join_none` in the function, where a separate spawned process executes the task.

Q3. If task A calls task B and B waits for 3 clock cycles, how long is A's caller blocked? For the whole of A, including the 3 cycles of B. Blocking time adds up through the call chain.

Q4. What happens when a task argument has the same name as a module signal? Inside the task the name refers to the argument; the module signal is hidden. Assignments intended for the signal go to the argument instead.

Q5. Why is the reference model a function and the driver a task? The reference model computes the expected value from the inputs with no dependence on time. The driver must synchronise with the clock and hold values for a number of cycles.

Part 4: Arguments

Everything in this part applies equally to functions and tasks. Where the two differ, the text says so.

Two words are used throughout:

- The **formal** argument is the name in the subroutine declaration: `x` in `function int f(int x)`.
- The **actual** argument is what the caller supplies: `count` in `f(count)`.

4.1 Argument Directions: input, output, inout and ref

In plain English

Each argument has a direction that says which way the data travels.

Direction	Meaning in everyday terms
`input`	"Here is a value for you to use." A photocopy is handed to the subroutine.
`output`	"Fill this in and give it back when you have finished." A blank form.
`inout`	"Here is a value; update it and give it back when you have finished." A photocopy, returned and filed over the original at the end.
`ref`	"Work directly on my original." No copy at all.

Technical definition

Direction	At the call	During execution	At the return
`input`	Actual value is **copied in** to the formal	Formal is a local variable; changes stay local	Nothing copied back
`output`	Nothing copied in; the formal starts at the **default value of its type**	Formal is a local variable	Formal value is **copied out** to the actual
`inout`	Actual value is copied in	Formal is a local variable	Formal value is copied out to the actual
`ref`	The formal becomes another name (an alias) for the actual variable	Every read and write acts on the caller's variable **immediately**	Nothing to copy

`input`, `output` and `inout` are all forms of **pass by value** (copying). `ref` is **pass by reference**.

How it works internally

```
task automatic demo(input int in_v, output int out_v, inout int io_v, ref int ref_v);

Caller variables            Subroutine variables
----------------            --------------------
x  [ 1 ] ----copy at call----------> in_v  [ 1 ]        changes are lost

y  [ 50]                            out_v [ 0 ]        starts at default, NOT 50
   [ 77] <---copy at return--------- out_v [ 77]

z  [ 10] ----copy at call----------> io_v  [ 10]
   [ 20] <---copy at return--------- io_v  [ 20]

w  [ 5 ] <=========================> ref_v              same storage, no copy;
                                                       a write is visible at once
```

Default direction and type

The rules by which omitted directions and types are filled in produce several interview questions.

Rule	Example	Result
The first argument defaults to `input`	`task t(int a);`	`a` is `input int`
An argument with no direction inherits the direction of the previous argument	`task t(output int a, b);`	`b` is also `output int`
An argument with no type inherits the type of the previous argument, **if its direction is not written**	`task t(input int a, b);`	`b` is `input int`
An argument whose direction **is** written but whose type is not becomes `logic`	`task t(input int a, output b);`	`b` is `output logic` (1 bit)
First argument with no type at all	`function f(a);`	`a` is `input logic` (1 bit)

> **Interview trap:** `task automatic t(input int a, b, output int c, d);` Here `b` is an `input int` and `d` is an `output int`, as intended. But in `task automatic t(input int a, output c, d);` both `c` and `d` are 1-bit `logic`, because writing the direction resets the type to the default. Always write direction and type for every argument.

Complete example

tb_arg_directions.sv

```
module tb_arg_directions;

  task automatic demo(input int in_v, output int out_v, inout int io_v);
    $display("inside : in_v=%0d out_v=%0d io_v=%0d", in_v, out_v, io_v);
    in_v  = in_v + 100;      // modifies the local copy only
    out_v = 77;
    io_v  = io_v * 2;
  endtask

  int x, y, z;

  initial begin
    x = 1;  y = 50;  z = 10;
    $display("before : x=%0d y=%0d z=%0d", x, y, z);
    demo(x, y, z);
    $display("after  : x=%0d y=%0d z=%0d", x, y, z);
  end

endmodule
```

EXPECTED OUTPUT

```
before : x=1 y=50 z=10
inside : in_v=1 out_v=0 io_v=10
after  : x=1 y=77 z=20
```

Line by line

Line	Explanation
`inside` line	`in_v` received a copy of `x`. `out_v` is **0**, not 50: an output formal does not receive the caller's value; it starts at the default of `int`. `io_v` received a copy of `z`.
`in_v = in_v + 100;`	Legal: an input formal is an ordinary local variable. The caller's `x` is unaffected.
`after` line	`x` is still 1. `y` received 77 and `z` received 20 when the task returned.

Output arguments are copied back only at the return

In a function this detail is invisible, because a function returns at once. In a task that consumes time it matters a great deal.

tb_output_vs_ref.sv

```
module tb_output_vs_ref;

  int out_val, ref_val;

  task automatic count_out(output int v);
    for (int i = 1; i <= 3; i++) begin
      #10;
      v = i;
    end
  endtask

  task automatic count_ref(ref int v);
    for (int i = 1; i <= 3; i++) begin
      #10;
      v = i;
    end
  endtask

  initial fork
    count_out(out_val);
    count_ref(ref_val);
  join

  initial begin
    #15 $display("[%0t] out_val=%0d ref_val=%0d", $time, out_val, ref_val);
    #10 $display("[%0t] out_val=%0d ref_val=%0d", $time, out_val, ref_val);
    #10 $display("[%0t] out_val=%0d ref_val=%0d", $time, out_val, ref_val);
  end

endmodule
```

EXPECTED OUTPUT

```
[15] out_val=0 ref_val=1
[25] out_val=0 ref_val=2
[35] out_val=3 ref_val=3
```

Both tasks write 1, 2, 3 to their argument at times 10, 20 and 30. The caller's `ref_val` changes at each write. The caller's `out_val` stays 0 until `count_out` returns at time 30, and then jumps straight to 3. An observer never sees the values 1 and 2.

Summary of the four directions

Property	`input`	`output`	`inout`	`ref`
Copied in at call	Yes	No	Yes	No copy
Copied out at return	No	Yes	Yes	No copy
Initial value of the formal	The actual's value	Default of the type	The actual's value	It **is** the actual
Caller sees changes	Never	At return	At return	Immediately
Subroutine sees later changes made by the caller's side	No	No	No	Yes
Actual may be a constant or expression	Yes	No, must be assignable	No, must be assignable	No, must be a variable
Actual may be a net (`wire`)	Yes	Yes	Yes	No
Type compatibility	Assignment compatible (converted)	Assignment compatible	Assignment compatible	Must match exactly

Property	`input`	`output`	`inout`	`ref`
Allowed in static subroutines	Yes	Yes	Yes	No, automatic only
Cost for a large array	Whole array copied	Whole array copied	Copied twice	Nothing copied

> **Common mistake:** Reading an `output` argument inside the subroutine and expecting the caller's value. It holds the type's default. Use `inout` or `ref` when the old value is needed.

> **Common mistake:** Passing a `logic [7:0]` module signal to an `output` argument of a task that runs for many cycles and expecting the signal to follow the task's assignments cycle by cycle. It changes once, when the task returns.

> **Verification application:** `output` suits a result that is complete only when the task ends: `task read_reg(input bit [7:0] addr, output logic [31:0] data);`. `ref` suits a signal or status variable that must be observed or updated while the task is still running.

Interview questions

Q1. What are the argument directions in SystemVerilog? `input`, `output`, `inout` and `ref`. The first three pass by value with copy-in and/or copy-out; `ref` passes by reference.

Q2. What is the default direction and type of an argument? `input` and `logic`. An argument written without a direction or type inherits them from the previous argument.

Q3. What is the value of an `output` argument at the start of a task? The default value of its type (0 for two-state, X for four-state), not the value of the caller's variable.

Q4. When does the caller's variable change for an `output` argument? When the subroutine returns. Intermediate values assigned during a time-consuming task are not visible to the caller.

Q5. What is the difference between `inout` and `ref`? `inout` copies the value in at the call and out at the return; in between the subroutine works on a private copy. `ref` creates no copy: the subroutine works on the caller's variable directly, so changes are visible in both directions immediately.

4.2 Pass by Value

In plain English

Passing by value means the subroutine receives its own copy of the data. Whatever it does to the copy, the caller's original is untouched (for `input`), or is replaced only at the end (for `output` and `inout`). This is the default behaviour in SystemVerilog and it is the safe one: a subroutine cannot accidentally damage the caller's data.

What "copy" means for each kind of data

Actual argument	What is copied	Consequence
Scalar or vector (`int`, `logic [31:0]`)	The value	Cheap
Struct	Every member	Cheap for small structs
Fixed-size array	**Every element**	Expensive for large arrays, repeated on every call
Dynamic array, queue, associative array	**Every element**	Same, and the size is unbounded
`string`	The characters	Usually small
Class handle	**The handle only**, not the object	Cheap. Caller and subroutine refer to the same object (see below)

Complete example

tb_pass_by_value.sv

```
module tb_pass_by_value;

  int data [4] = '{1, 2, 3, 4};
  int n;

  function automatic void try_change(int v);
    v = 99;                                  // changes the copy
  endfunction

  function automatic void clear_copy(int arr [4]);
    foreach (arr[i]) arr[i] = 0;             // clears the copy
  endfunction

  function automatic void clear_ref(ref int arr [4]);
    foreach (arr[i]) arr[i] = 0;             // clears the caller's array
  endfunction

  function automatic int total(const ref int arr [4]);
    int sum;
    sum = 0;
    foreach (arr[i]) sum += arr[i];
    return sum;
  endfunction

  function automatic void show(string label);
    $display("%s: %0d %0d %0d %0d  total=%0d", label,
             data[0], data[1], data[2], data[3], total(data));
  endfunction

  initial begin
    n = 5;
    try_change(n);
    $display("n after try_change = %0d", n);

    show("initial         ");
    clear_copy(data);
    show("after clear_copy");
    clear_ref(data);
    show("after clear_ref ");
  end

endmodule
```

EXPECTED OUTPUT

```
n after try_change = 5
initial         : 1 2 3 4  total=10
after clear_copy: 1 2 3 4  total=10
after clear_ref : 0 0 0 0  total=0
```

Line by line

Line	Explanation
`try_change(n);`	`v` is a copy of `n`. Assigning 99 to it changes nothing outside.
`clear_copy(int arr [4])`	The whole array is copied at the call. The function clears its copy; `data` is unchanged, and the copy is discarded. The function is useless as written, and it paid for copying four elements.
`clear_ref(ref int arr [4])`	No copy. `arr` is `data` under another name, so the caller's array is cleared.
`total(const ref int arr [4])`	Reads the caller's array without copying it and without being allowed to modify it. Section 4.3 explains `const ref`.

Class handles: passed by value, but the object is shared

This is a preview of Module 4, and one of the most frequently asked questions about arguments.

A variable of a class type does not contain an object; it contains a **handle**, which refers to an object. Passing it by value copies the handle. The copy refers to the same object.

```
   caller:      txn  ----------------+
                                     v
                                +--------+
                                | object |
                                +--------+
                                     ^
   subroutine:  t (copy of handle) --+

   t.a = 5;       modifies the shared object       -> the caller sees it
   t = new();     points the COPY at a new object  -> the caller's txn is unchanged
```

```
function automatic void set_a(alu_txn t);        // input handle
  t.a = 8'd5;            // changes the caller's object: both handles point to it
endfunction

function automatic void make_bad(alu_txn t);     // input handle
  t = new();             // only the local copy of the handle changes; caller's stays null
endfunction

function automatic void make_good(ref alu_txn t);
  t = new();             // the caller's handle now points to the new object
endfunction
```

So: to modify the **object**, an `input` handle is enough. To make the caller's **handle** point somewhere else, use `ref` or `output`, or return the handle.

> **Interview trap:** "Are class objects passed by value or by reference?" The precise answer: the *handle* is passed by value, and since a handle is a reference to the object, the object is effectively shared. Changes to the object's properties are seen by the caller. A new assignment to the handle inside the subroutine is not.

> **Common mistake:** Passing a large queue or array as an `input` to a function that is called once per transaction, for example `function bit is_in(int list[$], int x)`. The whole queue is copied on every call. Use `const ref`.

Interview questions

Q1. What is pass by value? The value of the actual argument is copied into the formal argument, which is a local variable of the subroutine. Changes to the formal do not affect the actual, except through the copy-out of `output` and `inout` arguments when the subroutine returns.

Q2. What is the default argument-passing mechanism in SystemVerilog? Pass by value, with direction `input`.

Q3. What happens when an array is passed as an `input`? The entire array is copied on each call. This costs time and memory, and any changes made inside are lost.

Q4. If a class handle is passed as an `input` and the function changes a property of the object, does the caller see the change? Yes. The handle is copied, but both handles refer to the same object.

Q5. If the function does `t = new();` on an input handle, does the caller's handle change? No. Only the local copy of the handle is redirected. Use `ref`, `output`, or a return value.

4.3 Pass by Reference Using ref

In plain English

With `ref`, the subroutine is not given a copy. It is given access to the caller's own variable. Inside the subroutine the formal name and the caller's variable are the same piece of storage. Writing to one is writing to the other, immediately, in both directions.

Technical definition

An argument declared `ref` is passed by reference: the formal argument becomes an alias of the actual argument for the duration of the call. No value is copied at the call or at the return.

`const ref` passes by reference but makes the formal read-only inside the subroutine. Any attempt to assign to it is a compile error.

Why it is needed

Need	How `ref` solves it
Efficiency: avoid copying a large array, queue or struct on every call	Nothing is copied
The subroutine must modify the caller's array or struct in place	Writes go to the original
A time-consuming task must see changes that happen **while it is running**	The task reads the live variable, not a snapshot taken at the call
A time-consuming task must make its updates visible **before it returns**	Writes are visible at once (Section 4.1, `tb_output_vs_ref`)

Need	How `ref` solves it
Several values must be returned or updated	Any number of `ref` arguments

Syntax

```
function automatic void scale(ref int arr[], input int factor);

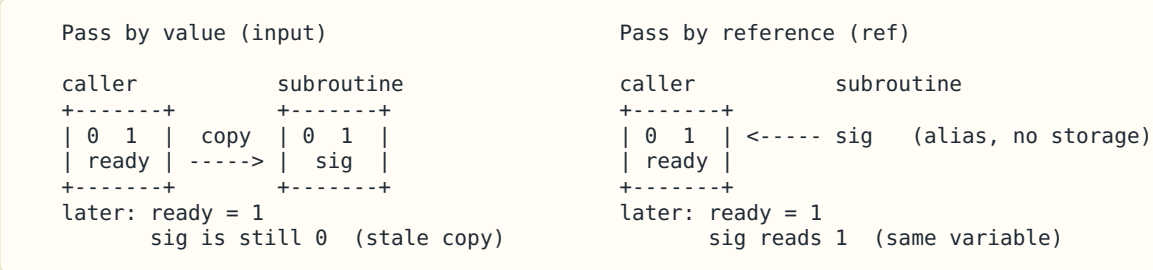
function automatic int  find_max(const ref int arr[]);      // read-only, no copy

task automatic wait_for(ref logic sig, input int max_cycles, output bit timed_out);
```

The rules of ref

Rule	Detail
Automatic subroutines only	A `ref` argument is illegal in a subroutine with static lifetime. In a module, write `task automatic` / `function automatic`. Class methods are automatic already
The actual must be a variable	A variable, a class property, a member of an unpacked struct, or an element of an unpacked array. Not a constant, not an expression, **not a net**
The type must match exactly	No implicit conversion: an `int` formal needs an `int` actual; a `logic [7:0]` actual cannot be bound to a `ref int`
`const ref` is read-only	Assignment to the formal is a compile error
Not inside `fork`...`join_any` / `join_none`	A `ref` formal may not be used inside a `fork`...`join_any` or `fork`...`join_none` block in the subroutine, because the subroutine might return (and the reference end) while the spawned process is still running
Direction is sticky	`ref` applies to following arguments until another direction is written: in `(ref int a, int b)`, `b` is also `ref`

How it works internally



Complete example: a task that must see a live signal

tb_ref_wait.sv

```
module tb_ref_wait;

  logic clk   = 0;
  logic ready = 0;
  bit   timed_out;

  always #5 clk = ~clk;

  initial #42 ready = 1;

  // WRONG: sig is a copy of the signal taken when the task was called
  task automatic wait_by_value(input logic sig, input int max_cycles, output bit timeout);
```

```
    timeout = 1;
    for (int i = 1; i <= max_cycles; i++) begin
      @(posedge clk);
      if (sig) begin
        $display("[%0t] by value: signal seen after %0d cycles", $time, i);
        timeout = 0;
        return;
      end
    end
    $display("[%0t] by value: timed out after %0d cycles", $time, max_cycles);
  endtask

  // RIGHT: sig is the caller's signal itself
  task automatic wait_by_ref(ref logic sig, input int max_cycles, output bit timeout);
    timeout = 1;
    for (int i = 1; i <= max_cycles; i++) begin
      @(posedge clk);
      if (sig) begin
        $display("[%0t] by ref  : signal seen after %0d cycles", $time, i);
        timeout = 0;
        return;
      end
    end
    $display("[%0t] by ref  : timed out after %0d cycles", $time, max_cycles);
  endtask

  initial begin
    wait_by_value(ready, 6, timed_out);
    $display("        timed_out = %b", timed_out);
    wait_by_ref(ready, 6, timed_out);
    $display("        timed_out = %b", timed_out);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[55] by value: timed out after 6 cycles
        timed_out = 1
[65] by ref  : signal seen after 1 cycles
        timed_out = 0
```

Execution flow

Time	What happens
0	`wait_by_value` is called. `ready` is 0, and that 0 is copied into `sig`.
5, 15, 25, 35	Four clock edges; `sig` is 0, and so is the real signal.
42	`ready` becomes 1 in the module. The task's `sig` is a private copy and stays 0.
45, 55	The fifth and sixth edges. The task still reads 0, although the real signal has been 1 since 42. After the sixth test the loop ends and the time-out is reported at 55.
55	`wait_by_ref` is called. Its `sig` is the module's `ready` itself.
65	The first `@(posedge clk)` waits for the next edge. `sig` reads 1: the signal is seen after one cycle.

When predicting times for code like this, list the clock edges explicitly (5, 15, 25...). Being one period off is the most common error in "at what time does this print?" questions.

Line by line

Line	Explanation
`input logic sig`	The value of `ready` at the moment of the call is copied. The task can never observe a later change. A task that polls an `input` argument is always a bug.
`ref logic sig`	The task reads the module's `ready` each time `sig` is evaluated.
`output bit timeout`	The status is needed only when the task ends, so `output` is appropriate. It is assigned before every exit path.
`return;`	Leaves the task early; the output is still copied back.

const ref

```
function automatic int find_max(const ref int arr[]);
  int m;
  m = arr[0];
  foreach (arr[i]) if (arr[i] > m) m = arr[i];
  // arr[0] = 0;        // compile error: arr is read-only
  return m;
endfunction
```

`const ref` gives the efficiency of a reference with the safety of an input. It is the right choice for any large array, queue or struct that a subroutine only reads.

ref compared with the other mechanisms

Property	`input`	`inout`	`ref`	`const ref`
Copy made	In	In and out	None	None
Subroutine can modify the caller's data	No	Yes, at return	Yes, immediately	No
Subroutine sees external changes during a time-consuming task	No	No	Yes	Yes
Works in static subroutines	Yes	Yes	No	No
Actual can be an expression or constant	Yes	No	No	No
Actual can be a net	Yes	Yes	No	No
Typical use	Small values	Small values to be updated	Large or live data to be modified	Large data to be read

Interview trap: "What is the difference between `inout` and `ref`? They both let the subroutine change the caller's variable." `inout` works on a private copy and writes it back at the return. `ref` works on the original throughout. The difference is visible when the subroutine takes time, or when the same variable is passed twice: with `task swap_add(inout int a, inout int b)` called as `swap_add(x, x)`, two separate copies exist and the last one copied out wins; with `ref` both names are the same variable.

Common mistake: `task wait_until_set(input bit flag); wait (flag); endtask`. If `flag` is 0 at the call, the task waits for ever, because its copy never changes. Declare the argument `ref` (and the task `automatic`).

> **Common mistake:** Declaring a `ref` argument in a module-level task without `automatic`. The compiler rejects it, because static subroutines cannot have reference arguments.

> **Common mistake:** Passing a `wire` or an expression such as `a + 1`, a constant, or a part-select of a vector to a `ref` argument. The actual must be a whole variable (or a struct member or unpacked array element) of exactly the formal's type.

> **Verification application:** Scoreboards and checkers pass queues of expected transactions to helper functions as `ref` (to pop from them) or `const ref` (to search them). Utility tasks that wait on a flag or counter owned by another component take it by `ref`. In class-based testbenches the need is reduced, because objects are already shared through handles and signals are reached through the virtual interface `vif`.

Interview questions

Q1. What is pass by reference? The formal argument becomes an alias for the caller's variable. No copy is made; every read and write in the subroutine acts directly on the caller's storage.

Q2. Why use `ref`? To avoid copying large arrays or structs, to modify the caller's data in place, and to let a time-consuming task see and make changes while it is running.

Q3. What is `const ref`? Pass by reference with a read-only formal. It avoids the copy but guarantees that the subroutine cannot change the caller's data.

Q4. What are the restrictions on `ref` arguments? The subroutine must be automatic; the actual must be a variable (not a net, constant or expression) whose type matches the formal exactly; and the formal cannot be used inside a `fork`...`join_any` or `fork`...`join_none` block.

Q5. Why is `ref` not allowed in static tasks? A static task has one permanent set of argument storage shared by all calls, and a reference to a caller's variable (which may itself be automatic and short-lived) cannot be stored safely there. The standard therefore restricts `ref` to automatic subroutines, where the reference exists only for the life of the call.

Q6. A task has `input logic sig` and loops `@(posedge clk); if (sig) ...`. What is wrong? `sig` is a copy made at the call and never changes. The task must take the signal by `ref`, or read it through an interface handle.

4.4 Default Argument Values and Binding by Name

In plain English

Many subroutines have arguments that are nearly always given the same value: a verbosity level, a number of cycles, a tag for messages. A **default value** lets the caller leave such an argument out. **Binding by name** lets the caller say which argument each value is for, so the order does not matter and the call documents itself.

Syntax

```
function automatic void report(string msg, int verbosity = 1, string tag = "TB");

report("one");                        // both defaults used
report("two", 3);                     // tag defaults
report("three", 2, "DRV");            // no defaults
report("four", , "MON");              // verbosity skipped with an empty position
report(.tag("SCB"), .msg("five"));    // by name, any order
report("six", .tag("GEN"));           // positional first, then by name
```

Rules

Rule	Detail
Any argument can have a default	Written as `= expression` after the name in the declaration
Omitted trailing arguments take their defaults	`report("x")`
A middle argument is skipped with an empty position	`report("x", , "MON")`
An argument without a default must be supplied	Otherwise a compile error
Named binding uses `.formal(actual)`	Same syntax as module port connections
Positional and named may be mixed	All positional arguments must come first
The default expression is evaluated at each call	In the scope where the subroutine is declared, not where it is called
Defaults are normally for `input` arguments	`inout` and `ref` defaults are permitted by the standard but unusual; `output` arguments are simply left unconnected when not needed

Complete example

tb_default_args.sv

```
module tb_default_args;

  logic clk = 0;
  always #5 clk = ~clk;

  function automatic void report(string msg, int verbosity = 1, string tag = "TB");
    $display("[%0t] [%s] (v%0d) %s", $time, tag, verbosity, msg);
  endfunction

  task automatic wait_cycles(int n = 1);
    repeat (n) @(posedge clk);
  endtask

  initial begin
    report("one");
    report("two", 3);
    report("three", 2, "DRV");
    report("four", , "MON");
    report(.tag("SCB"), .msg("five"));
    report("six", .tag("GEN"));

    wait_cycles();          // default: 1 cycle
    report("after wait_cycles()");
    wait_cycles(3);
    report("after wait_cycles(3)");
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[0] [TB] (v1) one
[0] [TB] (v3) two
[0] [DRV] (v2) three
[0] [MON] (v1) four
[0] [SCB] (v1) five
[0] [GEN] (v1) six
[5] [TB] (v1) after wait_cycles()
[35] [TB] (v1) after wait_cycles(3)
```

Line by line

Line	Explanation
`int verbosity = 1, string tag = "TB"`	Two arguments with defaults. `msg` has none and is mandatory.
`report("four", , "MON");`	The empty second position selects the default for `verbosity`.
`report(.tag("SCB"), .msg("five"));`	Named binding: order is free, and `verbosity` is simply not mentioned.
`wait_cycles();`	Uses `n` = 1: returns at the edge at time 5.
`wait_cycles(3);`	Edges at 15, 25 and 35.

Why this matters in real testbenches

- **Backward compatibility.** A new argument with a default can be added to a widely used task without editing any existing call.
- **Constructors.** `function new(string name = "alu_txn");` is the standard UVM constructor form. A constructor whose arguments all have defaults can be called as `new()`.

- **Readable calls.** `write_reg(.addr(8'h10), .data(32'h1), .check(0));` cannot be misread, whereas `write_reg(8'h10, 32'h1, 0);` relies on remembering the order.

> **Common mistake:** Changing the order or meaning of positional arguments in a shared task. Every positional call silently passes its values to the wrong arguments if the types happen to be compatible. Named binding protects against this.

> **Interview trap:** "A function is declared `f(int a, int b = 5, int c)`. Is `f(1, , 3)` legal? Is `f(1)`?" The first is legal: `b` takes its default. The second is an error: `c` has no default and was not supplied.

Interview questions

Q1. What are default argument values? Values written in the declaration (`int n = 1`) that are used when the caller omits that argument.

Q2. How do you skip an argument in the middle of the list? Leave its position empty, `f(1, , 3)`, or use named binding, `f(.a(1), .c(3))`.

Q3. What is argument binding by name? Supplying arguments as `.formal_name(value)`. The order then does not matter and omitted arguments take their defaults.

Q4. Can positional and named arguments be mixed in one call? Yes, but all positional arguments must come before the named ones.

4.5 Passing Arrays, Strings and Structs

Declaring array arguments

The formal argument is declared like a variable of the array type.

```
function automatic int sum_fixed (int arr [4]);            // fixed-size, exactly 4 elements
function automatic int sum_dyn   (int arr []);             // dynamic array formal
function automatic int sum_queue (int q [$]);              // queue formal
function automatic int count_keys(int aa [string]);        // associative array formal
function automatic void fill     (ref bit [7:0] mem [256], input bit [7:0] value);
```

Formal	Accepted actuals (pass by value)
Fixed-size `int arr [4]`	An array of the same element type and the same size
Dynamic `int arr []`	A dynamic array, a queue, or a fixed-size one-dimensional array with the same element type; the formal takes the size of the actual
Queue `int q [$]`	A queue, dynamic array or fixed-size one-dimensional array with the same element type
Associative `int aa [string]`	An associative array with the same element and index types

For `ref` arguments the types must match exactly, so a fixed-size array cannot be passed to a `ref int arr[]` formal.

Complete example

tb_array_args.sv

```
module tb_array_args;

  typedef struct {
    int       id;
    bit [7:0] payload [4];
  } pkt_t;

  // const ref: no copy, read-only
  function automatic int sum(const ref int arr[]);
    int s;
    s = 0;
    foreach (arr[i]) s += arr[i];
    return s;
  endfunction

  // ref: modifies the caller's array in place
  function automatic void scale(ref int arr[], input int factor);
    foreach (arr[i]) arr[i] *= factor;
  endfunction

  // input queue: a private copy that the function may consume freely
  function automatic int last_two_sum(int q[$]);
    int s;
    s  = q.pop_back();
    s += q.pop_back();
    return s;
  endfunction

  // struct by ref, string by value
  function automatic void stamp(ref pkt_t p, input string who);
    p.id++;
    foreach (p.payload[i]) p.payload[i] = p.payload[i] + 8'h10;
    $display("%s stamped packet %0d", who, p.id);
  endfunction

  int   values[];
  int   numbers[$] = {10, 20, 30, 40};
  pkt_t pkt;

  initial begin
    values = new[4];
    values = '{1, 2, 3, 4};

    $display("1. sum            = %0d", sum(values));
    scale(values, 10);
    $display("2. after scale    = %0d %0d %0d %0d",
             values[0], values[1], values[2], values[3]);
    $display("3. sum            = %0d", sum(values));

    $display("4. last_two_sum   = %0d", last_two_sum(numbers));
    $display("5. queue size is still %0d", numbers.size());

    pkt.id      = 6;
    pkt.payload = '{8'h01, 8'h02, 8'h03, 8'h04};
    stamp(pkt, "driver");
    $display("6. payload[0] = %h, payload[3] = %h", pkt.payload[0], pkt.payload[3]);
  end

endmodule
```

EXPECTED OUTPUT

```
1. sum           = 10
2. after scale   = 10 20 30 40
3. sum           = 100
4. last_two_sum  = 70
5. queue size is still 4
driver stamped packet 7
6. payload[0] = 11, payload[3] = 14
```

Line by line

Line	Explanation
`sum(const ref int arr[])`	The dynamic array is not copied and cannot be changed.
`scale(ref int arr[], input int factor)`	`input` is written explicitly before `factor`. Without it, `factor` would inherit `ref` from the previous argument and the literal `10` could not be passed.
`last_two_sum(int q[$])`	Passed by value. The function pops two elements from **its copy** (40 and 30). The caller's queue still has four elements.
`stamp(ref pkt_t p, input string who)`	The struct is modified in place; a struct passed as `input` would be copied and the changes lost.

> **Common mistake:** Forgetting that `ref` is inherited by the arguments that follow. `function void scale(ref int arr[], int factor)` makes `factor` a `ref int`, so `scale(values, 10)` fails to compile because a constant cannot be passed by reference.

> **Common mistake:** Writing an array formal as `int arr` or `int[] arr`. The dimension follows the name: `int arr[]`. To return an array, use a `typedef` (Module 1, Section 3.6).

Interview questions

Q1. How do you pass an array to a function without copying it? Declare the argument `ref` (or `const ref` if the function only reads it). The function must be automatic.

Q2. A function receives a queue as `input` and deletes elements from it. Does the caller's queue change? No. The function has its own copy.

Q3. In `function void f(ref int a[], int n);`, what is the direction of `n`? `ref`, inherited from the previous argument. Write `input int n` to make it an input.

Q4. What is the most efficient way to pass a 1 MB memory array to a read-only checking function? `const ref`.

Part 5: Lifetime, Recursion and Class Methods

5.1 Automatic and Static Lifetime

In plain English

Every variable needs somewhere to live. The question of **lifetime** is: when is that storage created, and when is it thrown away?

- A **static** variable is created once, before the simulation starts, and exists until the simulation ends. There is exactly one copy. Whatever value it was left with after one call of a subroutine is still there at the next call.
- An **automatic** variable is created each time execution enters the subroutine (or block) that declares it, and is discarded when execution leaves. Each call gets its own fresh copy.

Think of a meeting room. Static is a single whiteboard fixed to the wall: every group that uses the room sees what the last group wrote, and two groups in the room at the same time write over each other. Automatic is a fresh notepad handed to each group as it enters and collected as it leaves.

Technical definition

	Static lifetime	Automatic lifetime
Storage allocated	Once, at elaboration (before time 0)	On each entry to the task, function or block
Storage released	At the end of simulation	When that invocation ends (and no spawned process still uses it)
Number of copies	One, shared by all calls and all calling processes	One per active invocation
Initialiser in the declaration	Executed once, before time 0	Executed on every entry
Value at the start of a call	Whatever the previous call left	The initialiser, or the type's default

Lifetime applies to the **arguments** of a subroutine as well as to its local variables. In a static task the formal arguments are a single shared set of variables.

Why both exist

Verilog was designed to describe hardware, and hardware is static: a flip-flop exists for ever. So Verilog made everything static, including the variables inside tasks and functions. That choice is wrong for software-style code, where a subroutine may be active several times at once (called from parallel processes, or calling itself). SystemVerilog added automatic lifetime for that purpose and made it the only option inside classes, but kept static as the default in modules for backward compatibility.

The defaults

Where the subroutine or variable is declared	Default lifetime	Can it be changed?
Task or function in a `module`, `interface`, `program` or `package`	**Static**	Yes: `task automatic`, `function automatic`
All tasks and functions of a module, interface, program or package at once	Static	Yes: `module automatic m;`, `program automatic p;`, `package automatic pkg;`
Task or function in a `class` (a method)	**Automatic**	No. Class methods cannot have static lifetime
Variable declared directly in a module, interface, program or package	Static	No
Variable declared in an `initial` or `always` block of a module	Static	Yes: `automatic int x;`
Local variable or argument of a static subroutine	Static	Locals: yes, with `automatic`. Arguments follow the subroutine
Local variable of an automatic subroutine	Automatic	Yes: `static int x;`
Loop variable declared in the header: `for (int i = 0; ...)`	Automatic	—
Class property	Created with each object	`static` makes one copy for the whole class (Module 4)

Syntax

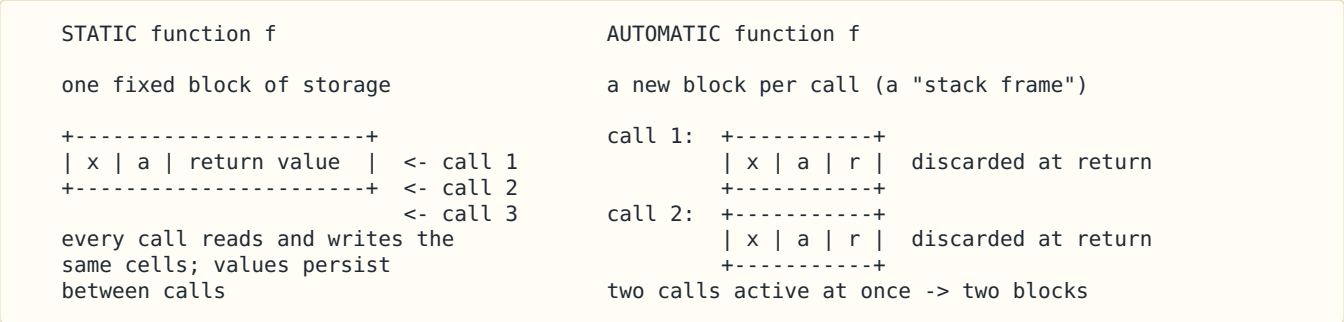
```
function automatic int f(int x);      // lifetime keyword goes after "function" / "task"
  int        a;                       // automatic (follows the function)
  static int calls = 0;               // explicitly static inside an automatic function
  ...
endfunction

task static t();                      // explicitly static (same as the default in a module)
  int           a;                    // static
  automatic int b = 0;                // explicitly automatic inside a static task
  ...
endtask

module automatic tb;                  // every task/function in this module is automatic
  ...
endmodule
```

Key point: The explicit-keyword rule from Section 1.2 applies here. In a static subroutine or block, a variable declaration **with an initialiser** must say `static` or `automatic` explicitly. `int count = 0;` on its own inside a static task is flagged by strict tools, because it is ambiguous to the reader: is it reset on each call (no) or set once before time 0 (yes)?

How it works internally



Complete example

tb_lifetime.sv

```systemverilog
module tb_lifetime;

  // Automatic function, automatic local: re-created and re-initialised on each call
  function automatic int count_auto();
    int n = 0;
    n++;
    return n;
  endfunction

  // Automatic function with an explicitly static local: one copy, initialised once
  function automatic int count_static_var();
    static int n = 0;
    n++;
    return n;
  endfunction

  // Static function (the module default): its locals are static
  function int count_static_func();
    int n;                  // static, starts at 0, keeps its value between calls
    n++;
    return n;
  endfunction

  initial begin
    $display("call  auto  static_var  static_func");
    for (int i = 1; i <= 3; i++)
      $display("%0d     %0d     %0d           %0d",
               i, count_auto(), count_static_var(), count_static_func());
  end

endmodule
```

EXPECTED OUTPUT

```
call  auto  static_var  static_func
1     1     1           1
2     1     2           2
3     1     3           3
```

Line by line

Line	Explanation
`count_auto`	`n` is created and set to 0 on every call, incremented, returned and destroyed. Always 1.
`static int n = 0;` in `count_static_var`	One copy for the whole simulation; the `= 0` is executed once before time 0. The value survives between calls: 1, 2, 3.

Line	Explanation
`function int count_static_func();`	No `automatic`, so the function and its local `n` are static. Same behaviour as the explicit `static`. The declaration has no initialiser, which avoids the explicit-keyword rule.
`for (int i = 1; ...)`	The loop variable is automatic regardless of the surrounding block.

What automatic variables cannot do

Restriction	Reason
Cannot be assigned with a nonblocking assignment (`<=`)	The update happens later in the time step, when the variable may no longer exist
Cannot be driven by a continuous assignment or used in `force`	Same reason
Cannot be referenced hierarchically (`tb.my_task.x`)	There may be zero or many copies
Cannot be used in `$monitor` or dumped to a waveform by `$dumpvars`	Same reason. This makes automatic variables harder to see in a wave viewer
An automatic subroutine's variables do not hold values between calls	By definition; use a `static` local or a module/class variable when persistence is wanted

Static versus automatic

Property	Static	Automatic
Copies	One	One per call
Keeps its value between calls	Yes	No
Safe when called from parallel processes	No	Yes
Recursion	Does not work	Works
`ref` arguments	Not allowed	Allowed
Nonblocking assignment to locals	Allowed	Not allowed
Visible in waveforms	Yes	Usually not
Default in modules, interfaces, programs, packages	Yes	
Default in classes		Yes (the only choice)
Models hardware storage	Yes	No

The keyword static has two meanings

Inside a class, `static` in front of a method or property means "belongs to the class, not to an object" (Module 4). It says nothing about lifetime.

Declaration (inside a class)	Meaning
`static function int get_count();`	A **static method**: can be called without an object as `alu_txn::get_count()`. Its lifetime is automatic, like every class method
`function static int f();`	A method with **static lifetime**: illegal in a class

Declaration (inside a class)	Meaning
`static int count;`	A static property: one copy shared by all objects

The position of the keyword tells them apart: before `function` / `task` it is the class qualifier; after it is the lifetime.

Interview trap: "What is the default lifetime of a task in a module? In a class? In a package?" Static, automatic, static. Functions declared in a package are static unless marked `automatic`, even though packages are mostly used by class-based code.

Common mistake: Assuming `int count = 0;` at the top of a module-level task resets `count` on each call. In a static task the initialisation happens once, before simulation starts. The counter keeps increasing across calls.

Verification application: Testbench code should be automatic everywhere: declare `module automatic tb_top;` or write `automatic` on each subroutine, and declare utility packages as `package automatic`. Class-based code gets this for free. Deliberate static locals are used for things that must persist, such as a unique-ID counter: `static int unsigned next_id = 0; return next_id++;`.

Interview questions

Q1. What is the difference between static and automatic lifetime? A static variable is allocated once and exists for the whole simulation, so all calls share it and it keeps its value between calls. An automatic variable is allocated on each entry to its subroutine or block and released on exit, so every call has its own copy.

Q2. What is the default lifetime of tasks and functions? Static in modules, interfaces, programs and packages. Automatic in classes.

Q3. How do you make all subroutines in a module automatic? Declare the module as `module automatic name;`. The same works for `program`, `interface` and `package`.

Q4. Can an automatic function contain a static variable? Yes, by declaring it with the `static` keyword. It is shared by all calls and initialised once.

Q5. Why can a nonblocking assignment not be made to an automatic variable? The assignment takes effect later, in the NBA region; by then the subroutine may have returned and the variable may no longer exist.

Q6. What is the difference between `static task foo();` and `task static foo();` in a class? The first declares a static method (called on the class, automatic lifetime). The second requests static lifetime, which is not allowed for class methods.

5.2 The Shared-Variable Bug in Static Tasks

In plain English

Because a static task has only one copy of its arguments and local variables, two processes that are inside the task at the same time are using the same variables. The second caller's arguments overwrite the first caller's. The first caller, when it wakes up from its delay, carries on with the second caller's values.

A function cannot be "inside" for any length of time, so two processes can never be in the same function simultaneously. The bug therefore needs a task with a timing control (or recursion, Section 5.3). It is the most asked question about lifetime.

Complete example

tb_static_task_bug.sv

```
module tb_static_task_bug;

  // Static by default: ONE copy of id and dly for all callers
  task send_static(input int id, input int dly);
    #dly;
    $display("[%0t] static   : finished id=%0d", $time, id);
  endtask

  // Automatic: each call has its own id and dly
  task automatic send_auto(input int id, input int dly);
    #dly;
    $display("[%0t] automatic: finished id=%0d", $time, id);
  endtask

  initial begin
    fork
      send_static(1, 10);
      send_static(2, 5);
    join

    fork
      send_auto(1, 10);
      send_auto(2, 5);
    join
  end

endmodule
```

EXPECTED OUTPUT

```
[5] static   : finished id=2
[10] static   : finished id=2
[15] automatic: finished id=2
[20] automatic: finished id=1
```

The output assumes that the two branches of each `fork` start in the order in which they are written, which is what the common simulators do. The standard does not guarantee the order; if the second branch started first, the static task would print `id=1` twice instead. Either way the static version reports the same ID twice.

What happened, step by step

```
   time 0   branch 1 calls send_static(1, 10)
              shared variables:   id = 1   dly = 10
              "#dly" is evaluated: wake up at time 10.  Branch 1 suspends.

   time 0   branch 2 calls send_static(2, 5)
              shared variables:   id = 2   dly = 5      <-- branch 1's values are overwritten
              "#dly" is evaluated: wake up at time 5.   Branch 2 suspends.

   time 5   branch 2 resumes:  prints id = 2            correct
   time 10  branch 1 resumes:  prints id = 2            WRONG, it was called with id = 1
```

With `send_auto`, each call has a private `id` and `dly`. Branch 1 keeps `id` = 1 for the whole call. The second `fork` starts at time 10, so its messages appear at 15 (branch 2, delay 5) and 20 (branch 1, delay 10).

Where this bug appears in practice

Situation	Symptom
A module-level BFM task (`write(addr, data)`) called from two parallel test threads	A transfer goes out with the other thread's address or data
A static task with a local loop counter, called concurrently	Loops end early or never, because both calls share the counter
A recursive function declared without `automatic`	Wrong result (Section 5.3)
A static task with `int count = 0;`	The counter is not reset on each call
Tasks in a package or interface used by class-based code	The same corruption, harder to find because the callers are in different classes

How to avoid it

1. Declare every testbench task and function `automatic`, or the whole module/package/program `automatic`.
2. Put testbench code in classes, where methods are always automatic.
3. Remember that `automatic` only protects the task's **own** arguments and locals. If two calls drive the same module signals or interface pins, they still collide. That is a resource-sharing problem and needs a semaphore (Module 7).

> **Interview trap:** "The task is automatic, so is it safe to call from two threads?" Only its local data is safe. If the task drives shared signals (`valid_in` , `a` , `b`), two simultaneous calls still fight over them. Automatic lifetime solves data corruption inside the task; mutual exclusion solves access to shared resources.

> **How to say it in an interview:** "In a module, tasks are static by default, so there is a single copy of the arguments and locals. If the task consumes time and is called from two concurrent processes, the second call overwrites the first call's arguments while the first is suspended, and the first resumes with the wrong values. Declaring the task `automatic` gives each call its own storage. Class methods are automatic by default, so this problem does not arise there."

Interview questions

Q1. What goes wrong when a static task with a delay is called from two parallel threads? Both calls use the same argument and local-variable storage. The later call overwrites the earlier call's values while the earlier call is waiting, so the earlier call continues with corrupted data.

Q2. Can the same problem occur with a static function? Not through concurrency, because a function completes in zero time and cannot be interrupted. It can occur through recursion, where the nested call overwrites the outer call's variables.

Q3. How do you fix it? Declare the task `automatic`.

Q4. Does `automatic` make a task fully re-entrant? It makes the task's arguments and locals private to each call. Module-level variables and signals that the task reads or writes are still shared and must be protected separately.

5.3 Recursive Functions

In plain English

A recursive function is a function that calls itself. It solves a problem by reducing it to a smaller version of the same problem, until the problem is so small that the answer is known directly. Factorial is the standard example: 5! is 5 × 4!, 4! is 4 × 3!, and so on, until 1!, which is 1.

Technical definition

Recursion is a subroutine invoking itself, directly or through other subroutines. A correct recursive subroutine has

1. a **base case**: a condition under which it returns without calling itself, and
2. a **recursive case**: a call to itself with an argument that moves towards the base case.

In SystemVerilog a recursive subroutine must be **automatic**. Each nested call needs its own copy of the arguments and locals, which is exactly what automatic lifetime provides. Tasks may be recursive as well as functions.

How it works internally

```
factorial(4)                      the "call stack": one frame per active call

factorial(4):  n = 4   needs 4 * factorial(3)   ---------+
  factorial(3):  n = 3   needs 3 * factorial(2)   ------+ |   going down:
    factorial(2):  n = 2   needs 2 * factorial(1)   --+ | |   frames are created
      factorial(1):  n = 1   base case: returns 1     | | |
    factorial(2) returns 2 * 1 = 2   <----------------+ | |   coming back up:
  factorial(3) returns 3 * 2 = 6   <--------------------+ |   frames are destroyed
factorial(4) returns 4 * 6 = 24  <------------------------+

With automatic lifetime there are four separate variables called n, one per frame.
With static lifetime there is ONE n: the inner calls overwrite it.
```

Complete example

tb_recursion.sv

```
module tb_recursion;

  function automatic longint unsigned factorial(int unsigned n);
    if (n <= 1) return 1;                       // base case
    return n * factorial(n - 1);                // recursive case
  endfunction

  function automatic int fib(int n);
    if (n < 2) return n;                        // fib(0) = 0, fib(1) = 1
    return fib(n - 1) + fib(n - 2);
  endfunction

  function automatic int gcd(int x, int y);
    if (y == 0) return x;
    return gcd(y, x % y);
  endfunction

  // Sum of 1..n, automatic: correct
  function automatic int sum_auto(int n);
    int partial;
    if (n == 0) return 0;
    partial = sum_auto(n - 1);
    return partial + n;
```

```
  endfunction

  // The same code with static lifetime: broken
  function int sum_static(int n);
    int partial;
    if (n == 0) return 0;
    partial = sum_static(n - 1);
    return partial + n;
  endfunction

  // A recursive task that consumes time
  task automatic countdown(int n);
    if (n == 0) begin
      $display("[%0t] lift-off", $time);
      return;
    end
    $display("[%0t] %0d", $time, n);
    #10;
    countdown(n - 1);
  endtask

  initial begin
    $display("factorial(5)  = %0d", factorial(5));
    $display("factorial(20) = %0d", factorial(20));
    $display("fib(10)       = %0d", fib(10));
    $display("gcd(48, 18)   = %0d", gcd(48, 18));
    $display("sum_auto(3)   = %0d", sum_auto(3));
    $display("sum_static(3) = %0d", sum_static(3));
    countdown(3);
  end

endmodule
```

EXPECTED OUTPUT

```
factorial(5)  = 120
factorial(20) = 2432902008176640000
fib(10)       = 55
gcd(48, 18)   = 6
sum_auto(3)   = 6
sum_static(3) = 0
[0] 3
[10] 2
[20] 1
[30] lift-off
```

Line by line

Line	Explanation
`function automatic longint unsigned factorial(...)`	The result grows fast: 13! already exceeds 32 bits, so the return type is 64-bit. 20! is the largest factorial that fits in 64 bits.
`if (n <= 1) return 1;`	The base case. Without it the function would call itself for ever.
`return n * factorial(n - 1);`	The function is suspended in the middle of this expression while the nested call runs; its own `n` is preserved in its own frame.
`fib`	Two recursive calls per invocation. Simple but exponentially slow; `fib(10)` already makes 177 calls.
`gcd(48, 18)`	Euclid's algorithm: gcd(48, 18) → gcd(18, 12) → gcd(12, 6) → gcd(6, 0) → 6.
`sum_auto(3)`	3 + 2 + 1 + 0 = 6.
`sum_static(3)`	The same statements give 0. See the trace below.
`countdown`	A recursive task. Each level waits 10 time units before calling the next, so the calls are spread over simulation time.

Why the static version returns 0

```
sum_static has ONE variable n and ONE variable partial.

sum_static(3):  n = 3;  calls sum_static(2)
  n = 2;  calls sum_static(1)
    n = 1;  calls sum_static(0)
      n = 0;  base case: returns 0
    back in the call that was made with 1:  partial = 0;  return partial + n
                                            but n is now 0  ->  returns 0
  back in the call that was made with 2:    partial = 0;  n is still 0  ->  returns 0
back in the call that was made with 3:      partial = 0;  n is still 0  ->  returns 0
```

Every level adds the shared `n`, which the innermost call left at 0. Some simulators warn about recursion in a static function; all of them compute a wrong answer.

Rules and cautions

Point	Detail
Must be `automatic`	In a module or package. Class methods are automatic already
Must have a reachable base case	Otherwise infinite recursion: the simulator runs out of stack and crashes or reports a stack overflow
Depth is limited	Each level uses stack memory; tens of thousands of levels may exceed the simulator's limit. Prefer a loop for long linear recursions
Zero time for functions	A recursive function still completes in zero time however deep it goes
Synthesis	Recursive functions are generally not synthesizable. Some tools accept recursion whose depth is fixed by constants at elaboration time
Indirect recursion	`a` calls `b` and `b` calls `a`: also recursion, also needs `automatic` on both

Recursion versus iteration

Property	Recursion	Loop
Code for naturally recursive problems (trees, divide and conquer, nested structures)	Short and clear	Needs an explicit stack
Speed and memory	A call and a frame per level	Usually faster, constant memory
Risk	Stack overflow, missing base case	Infinite loop
Synthesizable	Rarely	Yes, with constant bounds

Interview trap: "Write a factorial function." The most common errors are: forgetting `automatic`; forgetting the base case for 0; and returning `int`, which overflows at 13!. Mention all three even if you are not asked.

Verification application: Recursion appears in testbenches when walking nested data: printing a tree of components, flattening a hierarchical register map, generating all combinations of a set of options for exhaustive stimulus. UVM uses recursive calls to traverse the component hierarchy in each phase.

Interview questions

Q1. What is a recursive function and what does it need to work in SystemVerilog? A function that calls itself. It needs a base case that ends the recursion, and it must be declared `automatic` so that each call has its own arguments and local variables.

Q2. What happens if a recursive function is not declared automatic? All levels share one set of variables. Inner calls overwrite the values that outer calls still need, and the result is wrong.

Q3. Can a task be recursive? Yes, if it is automatic. Each level may consume time.

Q4. What happens if the base case is missing? The function calls itself without end until the simulator's stack is exhausted, causing a crash or a stack-overflow error.

Q5. Are recursive functions synthesizable? Generally no. Hardware cannot be created for a depth that is known only at run time.

Q6. Write a recursive function to compute the sum of the digits of a number. `function automatic int digit_sum(int unsigned n); if (n < 10) return n; return (n % 10) + digit_sum(n / 10); endfunction`

5.4 Function and Task Declarations Inside Classes

This section is a preview. Classes are the subject of Module 4; here the aim is only to connect what you have learned about functions and tasks to the place where they are used most.

In plain English

A class groups data together with the subroutines that work on that data. The data items are called **properties**; the subroutines are called **methods**. A method is an ordinary function or task, declared inside the class, that can read and write the properties of the object on which it is called.

What stays the same and what changes

Topic	In a module	In a class
Function versus task	Function: zero time. Task: may consume time	Identical
Argument directions, defaults, named binding	As in Part 4	Identical
Default lifetime	Static	**Automatic**, always
`automatic` keyword	Needed	Not needed
What the body can access	Module variables and signals	The object's own properties (directly, or through `this`)
How it is called	`my_task(args);`	`handle.method(args);`
Special subroutine	—	`new`, the constructor: a function with no return type
Extra qualifiers	—	`static`, `virtual`, `pure virtual`, `local`, `protected`, `extern`

Method qualifiers at a glance

Qualifier	Meaning	Module
(none)	Ordinary method, called through an object handle	4
`static`	Belongs to the class; called as `class_name::method()`; cannot access non-static properties	4
`virtual`	Can be overridden in a derived class; the version executed is chosen by the object's type at run time	4
`pure virtual`	Declared in an abstract class with no body; derived classes must implement it	4
`local`, `protected`	Restrict who may call the method	4
`extern`	The body is written outside the class as `function return_type class_name::method(...)`	4

Complete example

Compile with `alu_pkg.sv` from Module 1.

`tb_class_methods.sv`

```
module tb_class_methods;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0] a, b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    // Void function: prints the object. Zero time.
    function void print(string tag = "");
      $display("%s id=%0d op=%s a=%0d b=%0d result=%0d",
               tag, id, op.name(), a, b, result);
    endfunction

    // Function returning a handle to a new object with the same contents
    function alu_txn copy();
      alu_txn t;
      t        = new();
      t.a      = a;
      t.b      = b;
      t.op     = op;
      t.result = result;
      t.id     = id;
      return t;
    endfunction

    // Function returning a status bit; the argument is a handle to another object
    function bit compare(alu_txn rhs);
      return (a == rhs.a) && (b == rhs.b) && (op == rhs.op) && (result === rhs.result);
    endfunction

    // Body written outside the class
    extern function logic [8:0] expected();
  endclass

  function logic [8:0] alu_txn::expected();
    case (op)
      ALU_ADD: return {1'b0, a} + {1'b0, b};
      ALU_SUB: return {1'b0, a} - {1'b0, b};
      default: return 'x;        // remaining operations omitted in this preview
    endcase
  endfunction
```

```
  class ticker;
    string name;

    function new(string name = "ticker");     // constructor: a function, zero time
      this.name = name;                       // this.name is the property, name the argument
    endfunction

    task run(int n, int period);              // a task: consumes time
      repeat (n) begin
        #period;
        $display("[%0t] %s tick", $time, name);
      end
    endtask
  endclass

  alu_txn t1, t2;
  ticker  tk;

  initial begin
    t1 = new();
    t1.id     = 1;
    t1.a      = 8'd200;
    t1.b      = 8'd100;
    t1.op     = ALU_ADD;
    t1.result = t1.expected();
    t1.print("[GEN]");

    t2    = t1.copy();
    t2.id = 2;
    t2.print("[CPY]");
    $display("compare before change: %b", t1.compare(t2));

    t2.b = 8'd50;
    $display("compare after  change: %b", t1.compare(t2));

    tk = new("clk_a");
    tk.run(2, 10);
  end

endmodule
```

EXPECTED OUTPUT

```
[GEN] id=1 op=ALU_ADD a=200 b=100 result=300
[CPY] id=2 op=ALU_ADD a=200 b=100 result=300
compare before change: 1
compare after  change: 0
[10] clk_a tick
[20] clk_a tick
```

Line by line

Line	Explanation
`function void print(string tag = "");`	No `automatic` keyword: methods are automatic. The body uses `id`, `op`, `a`, `b`, `result` directly; they are the properties of the object the method was called on.
`t1.print("[GEN]");`	Calls `print` on the object that `t1` refers to.
`function alu_txn copy();`	Returns a class handle. `t` is declared and then assigned `new()`, which creates a second object; the fields are copied one by one.
`function bit compare(alu_txn rhs);`	`rhs` is an input handle to another object. `a` is this object's property; `rhs.a` is the other object's. `id` is deliberately not compared. `===` is used for the four-state `result`.
`extern function ... expected();`	Only the prototype is inside the class. The body follows the class and names its class with `alu_txn::`. This keeps a class declaration short enough to read as a summary.

Line	Explanation
`function new(string name = "ticker");`	The constructor. It has no return type, is called by `new(...)`, and being a function it cannot wait.
`this.name = name;`	The argument hides the property of the same name; `this.name` refers to the property.
`task run(int n, int period);`	A task method with delays. `tk.run(2, 10)` blocks the caller for 20 time units.
`t2 = t1.copy(); t2.id = 2;`	`t2` refers to a separate object, so changing `t2.id` and later `t2.b` does not affect `t1`.

Functions and tasks in the layered testbench

Component	Method	Function or task	Why
`alu_txn`	`new`, `print`, `copy`, `compare`	Functions	Operate on data only
`alu_generator`	`run`	Task	Blocks on `gen2drv_mbx.put()` and between transactions
`alu_driver`	`run`, `drive`	Tasks	Wait for mailbox items and clock edges
`alu_monitor`	`run`	Task	Waits for clock edges, samples `vif`
`alu_scoreboard`	`run`	Task	Blocks on `mon2scb_mbx.get()`
`alu_scoreboard`	`predict` / reference model, `report`	Functions	Pure calculation and printing
`alu_env`	`new`, `build` / `connect`	Functions	Create and connect objects in zero time
`alu_env`, `alu_test`	`run`	Tasks	Start the components with `fork` and wait for the test to finish

The rule from Part 3 holds throughout: a method is a task only when it has to wait.

> **Interview trap:** "Do you need to declare class methods `automatic`?" No. They are automatic by default and cannot be static-lifetime. "Then can the shared-variable bug happen in a class?" Not with the method's arguments or locals. It can still happen with class properties, since all calls on the same object share that object's properties.

> **Common mistake:** Putting time-consuming code in the constructor. `new` is a function, so `@(posedge vif.clk)` inside it is a compile error. Constructors only build and connect; waiting belongs in `run()`.

Interview questions

Q1. What is a method? A function or task declared inside a class. It operates on the properties of the object it is called on.

Q2. What is the default lifetime of class methods? Automatic. It cannot be changed to static.

Q3. Why is the constructor a function and not a task? Object creation must complete in zero time; `new` is used in expressions such as `t = new();`. Therefore it cannot contain timing controls.

Q4. In a testbench, which methods are tasks and which are functions? Methods that wait for a clock, a mailbox or another component (`run`, `drive`) are tasks. Methods that only compute, copy, compare, print or build (`new`, `copy`, `compare`, `print`, the reference model) are functions.

Q5. What does `extern` do? It lets the class contain only the method's prototype, with the body defined outside the class using the scope operator: `function void alu_txn::print(...)`.

Part 6: Module 3 Revision and Practice

6.1 Revision Summary

Procedural blocks

Block	Runs	Timing controls	Use
`initial`	Once, from time 0	Allowed	Stimulus, test start
`always`	For ever, from time 0	Required (or it hangs)	Clocks, legacy RTL, monitors
`always_comb`	At time 0 and on input change	Not allowed	Combinational logic, `=`
`always_ff`	On the clock/reset edge	Only the edge list	Flip-flops, `<=`
`always_latch`	At time 0 and on input change	Not allowed	Latches
`final`	Once, at the end	Not allowed	Summary report

- All blocks are concurrent processes. Order between blocks at the same time is undefined.
- A process runs without interruption until it reaches a timing control.

Assignments and regions

- `=` updates immediately; `<=` evaluates the right-hand side now and updates in the NBA region.
- Sequential logic: `<=`. Combinational logic and testbench variables: `=`. Never mix on one variable.
- Region order: Preponed → Active → Inactive → NBA → Observed → Reactive → Postponed.
- `$display` prints at once (Active). `$strobe` and `$monitor` print in Postponed, after NBA updates.
- `#0` moves a process to the Inactive region: later than active processes, earlier than NBA updates.

Functions and tasks

	Function	Task
Time	Zero, always	May consume time
`#`, `@`, `wait`, blocking calls	Illegal	Legal
Returns	One value or `void`	Nothing
In an expression	Yes	No
Calls a task	No (only inside `fork`…`join_none`)	Yes
Calls a function	Yes	Yes
Default return type	1-bit `logic`	—
Synthesizable	Yes	Only without timing

- Return with `return expr;` or by assigning to the function name; with neither, the type's default is returned.
- `void'(f())` discards a result deliberately.
- A subroutine that never waits should be a function (void if it returns nothing).

- A function cannot consume time because it is an operand of an expression, and expressions are evaluated atomically.

Arguments

Direction	Copy in	Copy out	Caller sees changes	Notes
`input` (default)	Yes	No	Never	Actual may be any expression
`output`	No	At return	At return	Formal starts at the type default
`inout`	Yes	At return	At return	
`ref`	—	—	Immediately	Automatic only; actual is a variable of the same type; not a net
`const ref`	—	—	Read-only	No copy, cannot be modified

- Default direction `input`, default type `logic`. Direction and type are inherited from the previous argument; writing a direction without a type gives `logic`.
- Arrays, queues and structs passed by value are copied in full. Use `ref` / `const ref`.
- A class handle passed by value still refers to the same object.
- Default values: `int n = 1`. Skip with an empty position or use `.name(value)` binding; positional arguments come first.

Lifetime

Declared in	Default lifetime
Module, interface, program, package	Static
Class	Automatic (fixed)

- Static: one copy, keeps its value, initialised once before time 0.
- Automatic: one copy per call, re-initialised on each entry.
- Static task + delay + two callers = corrupted arguments. Fix: `automatic`.
- Recursion requires `automatic` and a base case.
- `ref` requires `automatic`. Nonblocking assignment to an automatic variable is illegal.
- In a class, `static function` means a class-level method, not static lifetime.

One-line answers

Question	Answer
Can a function have a delay?	No
Can a function call a task?	No
Can a function have output arguments?	Yes
Can a function contain `fork` ... `join_none`?	Yes
Can a task return a value?	No, only through arguments

Question	Answer
Can a task be used in an expression?	No
Must a task consume time?	No
Default return type of a function?	1-bit `logic`
Default argument direction?	`input`
Default lifetime in a module?	Static
Default lifetime in a class?	Automatic
Is `new()` a function or a task?	A function
Are objects copied when passed to a method?	No, only the handle

6.2 Interview Questions and Answers

These are in addition to the questions at the end of each section.

Basic level

Q1. What is a subroutine and what kinds does SystemVerilog have? A named block of code that can be called from several places. SystemVerilog has functions (zero time, return a value) and tasks (may consume time, no return value).

Q2. Which keywords end a function and a task? `endfunction` and `endtask`.

Q3. How many values can a function return? One, through its return value. Additional values can be passed back through `output`, `inout` or `ref` arguments, or by returning a struct, array or class handle.

Q4. What is the difference between `return;` and `return expr;`? `return;` leaves a task or a void function. `return expr;` leaves a non-void function and sets its result.

Q5. Is `begin` ... `end` required inside a task or function? No. In SystemVerilog several statements may be written directly between the header and `endtask` / `endfunction`.

Q6. What does `automatic` mean? Each call of the subroutine gets its own freshly allocated copy of the arguments and local variables, which is released when the call ends.

Q7. Can a task have zero arguments? Can a function? Yes to both.

Q8. What is the difference between `$display` and `$strobe`? `$display` prints immediately. `$strobe` prints at the end of the current time step, after nonblocking assignments have been applied.

Q9. Where can a function be called from that a task cannot? From any expression, a continuous assignment, an `always_comb` block, a constraint, another function, a constructor and a `final` block.

Q10. What is the difference between a blocking and a nonblocking assignment in one sentence each? Blocking (`=`): the variable is updated before the next statement runs. Nonblocking (`<=`): the new value is computed now but the variable is updated at the end of the time step.

Q11. What does `void'(...)` do? It discards the return value of a function call so that the function can be used as a statement without a warning.

Q12. How do you pass an argument by reference? Declare the formal argument with the `ref` keyword in an automatic subroutine.

Intermediate level

Q13. Why can a task with no delay still not be called from a function? The language decides legality from the kind of subroutine, not from its body. Any task is allowed to consume time, so no task may be called from a function. If it does not need time, it should be declared as a void function.

Q14. A task has an `output` argument and takes 100 ns. When does the caller's variable change? At the end of the 100 ns, when the task returns. To see intermediate values, use a `ref` argument.

Q15. What is the difference between `task automatic` in a module and a task in a class? None in lifetime: both are automatic. The class task additionally has access to the object's properties through the implicit `this` handle and can be `virtual`.

Q16. What is wrong with `function int f(input int a, output b);`? `b` has a written direction but no type, so it is a 1-bit `logic`, not an `int`. Results wider than one bit are truncated.

Q17. How do you write a function that modifies a queue owned by the caller? `function automatic void f(ref int q[$]);`. With an `input` queue the function would modify a copy.

Q18. Why is `const ref` preferred over `input` for a large array? `input` copies the whole array on every call. `const ref` passes a reference, so nothing is copied, and the `const` prevents accidental modification.

Q19. A function has both `foo = 5;` (assignment to its name) and later `return 7;`. What is returned? 7. The `return` statement sets the result and ends the function.

Q20. Can `always_comb` call a function that reads module-level signals not passed as arguments? What about `always @*`? Yes. `always_comb` is sensitive to signals read inside the functions it calls. `always @*` is sensitive only to signals that appear in the block itself, so a change in a signal read only inside the function does not trigger it. This is one reason to use `always_comb`.

Q21. What is the difference between `disable` and `return` for ending a task? `return` ends the current invocation cleanly and copies outputs back. `disable task_name` terminates every active invocation of that task, from any process, and the values of its outputs are not guaranteed.

Q22. What happens to a `static` local variable in an `automatic` function? It is allocated once and shared by all calls; its initialiser runs once before time 0. It is used for things like call counters and ID generators.

Q23. Can default argument values depend on other variables? Yes. The default is an expression evaluated at each call, in the scope where the subroutine is declared.

Q24. Why do testbench drive tasks use nonblocking assignments or clocking blocks? The task and the DUT both act on the same clock edge. With a blocking assignment the DUT might sample either the old or the new value, depending on process order. A nonblocking assignment (or a clocking block drive) updates the signal after the DUT has sampled, removing the race.

Advanced and scenario-based

Q25. A bus-write task in an interface works when called from one test thread, but data is corrupted when two threads use it. Give two possible causes. (1) The task is static, so both calls share one set of arguments and locals; the second call overwrites the first while it is waiting. Fix: `automatic`. (2) Even with an automatic task,

both calls drive the same interface signals at the same time. Fix: protect the bus with a semaphore so that only one transfer is in progress.

Q26. A function is called millions of times and the simulation is slow. The profile points at the call itself. What do you look for? Large arguments passed by value: arrays, queues, associative arrays, structs or strings being copied on every call. Change them to `const ref` (or `ref`).

Q27. Why does the standard allow `fork ... join_none` in a function but not `fork ... join`? With `join` and `join_any` the parent waits for at least one child, which may take time. With `join_none` the parent continues immediately, so the function still returns in zero time; the children run later as separate processes.

Q28. Your scoreboard's `check()` was written as a task. A colleague wants to call it from a covergroup sample function. What is the problem and the fix? A function cannot call a task. `check()` does not need to wait, so change it to a void function.

Q29. Explain why `wait_for(input bit done)` can hang, and give two correct designs. The input is a copy taken at the call, so the task never sees `done` change. Correct designs: take the flag by `ref` in an automatic task, or let the task read the signal directly (through a virtual interface or because it is visible in the enclosing scope) instead of receiving it as an argument.

Q30. In a package you write `function int next_id(); int id; id++; return id; endfunction`. Is `id` reset on each call? No. Package functions are static by default, so `id` is static, starts at 0 once and keeps counting: 1, 2, 3... This happens to be the intended behaviour here, but it should be written explicitly as `function automatic int next_id(); static int id; ...` so that the intent is clear.

Q31. What is printed, and why? `initial begin a = 0; a <= 1; #0 $display(a); end` 0. `#0` reschedules the process in the Inactive region, which is processed before the NBA region, so the nonblocking update has not yet happened.

Q32. Why does UVM split a component's work into function phases and task phases? Phases that only construct and connect the testbench (`build_phase`, `connect_phase`) or report results must complete in zero time and in a fixed order, so they are functions. The `run_phase` drives and monitors the DUT over simulation time, so it is a task. It is the same function/task rule applied to a whole methodology.

Q33. Can a function modify a variable that is not one of its arguments? Yes. A function can read and write any variable visible in its scope (module variables, class properties). Such side effects are legal but make the function unusable as a constant function and can make `always_comb` behaviour confusing; prefer functions that depend only on their arguments in RTL.

Q34. Two `always @(posedge clk)` blocks each call the same static function with different arguments. Can they corrupt each other? No. Each call runs to completion without interruption, because a function cannot suspend. The second call starts only after the first has returned. Concurrency corruption needs a subroutine that suspends, which means a task.

Q35. A recursive automatic function works for small inputs and crashes the simulator for large ones. Why? Each level of recursion allocates a new frame. A very deep recursion exhausts the simulator's stack. Rewrite it as a loop, or reduce the depth (for example by halving the problem instead of decrementing it).

Q36. What is the difference between passing a struct and passing a class handle as an `input`? The struct is copied member by member; changes inside the subroutine are lost. The handle is copied but refers to the same object; changes to the object's properties are seen by the caller.

6.3 Output-Prediction Questions

Work out each answer on paper before reading it.

P1.

```
module p1;
  task automatic t(input int a, output int b, inout int c);
    b = b + a;
    c = c + a;
    a = 0;
  endtask
  int x = 3, y = 10, z = 20;
  initial begin
    t(x, y, z);
    $display("%0d %0d %0d", x, y, z);
  end
endmodule
```

Answer: `3 3 23`. `a` is a copy, so `x` stays 3. The output formal `b` starts at 0, not 10, so `b` = 0 + 3. The inout formal `c` starts at 20 and returns 23.

P2.

```
module p2;
  function automatic f(int a);
    return a + 1;
  endfunction
  initial $display("%0d %0d", f(4), f(5));
endmodule
```

Answer: `1 0`. No return type means 1-bit `logic`. 5 is `...101` → 1; 6 is `...110` → 0.

P3.

```
module p3;
  function int bump();
    int v;
    v += 2;
    return v;
  endfunction
  function automatic int bump_a();
    int v;
    v += 2;
    return v;
  endfunction
  initial begin
    void'(bump());   void'(bump());
    void'(bump_a()); void'(bump_a());
    $display("%0d %0d", bump(), bump_a());
  end
endmodule
```

Answer: `6 2`. `bump` is static: `v` persists and reaches 6 on the third call. `bump_a` is automatic: `v` starts at 0 on every call.

P4.

```
module p4;
  int a = 1;
  initial begin
    a <= a + 1;
    a  = a + 10;
    $display("%0d", a);
    $strobe ("%0d", a);
  end
endmodule
```

Answer: `11` then `2`. The nonblocking statement evaluates `a + 1` = 2 immediately and saves it. The blocking statement makes `a` 11, which `$display` prints. In the NBA region `a` becomes 2, which `$strobe` prints at the end of the time step. (Mixing the two assignment types on one variable is bad practice; the question tests the scheduling.)

P5.

```
module p5;
  logic clk = 0;
  always #5 clk = ~clk;
  task automatic t(int n);
    repeat (n) @(posedge clk);
    #2;
  endtask
  initial begin
    #7;
    t(2);  $display("%0t", $time);
    t(1);  $display("%0t", $time);
    $finish;
  end
endmodule
```

Answer: `27` then `37`. Rising edges are at 5, 15, 25, 35. From time 7 the next two edges are 15 and 25; plus 2 gives 27. From 27 the next edge is 35; plus 2 gives 37.

P6.

```
module p6;
  function automatic int f(int a = 1, int b = 2, int c = 3);
    return a * 100 + b * 10 + c;
  endfunction
  initial $display("%0d %0d %0d %0d", f(), f(5), f(, 7), f(.c(9), .a(4)));
endmodule
```

Answer: `123 523 173 429`.

P7.

```
module p7;
  function automatic void f1(int q[$]);      q.push_back(9); endfunction
  function automatic void f2(ref int q[$]);  q.push_back(9); endfunction
  int q[$] = {1, 2};
  initial begin
    f1(q);  $display("%0d", q.size());
    f2(q);  $display("%0d", q.size());
  end
endmodule
```

Answer: `2` then `3`. `f1` appends to a copy. `f2` appends to the caller's queue.

P8.

```
module p8;
  task automatic p(int n);
    if (n == 0) return;
    $write("%0d ", n);
    p(n - 1);
    $write("%0d ", n);
  endtask
  initial begin
    p(3);
    $display("");
  end
endmodule
```

Answer: `3 2 1 1 2 3`. Each level prints its `n`, recurses, and prints its own `n` again on the way back. Because the task is automatic, each level still has its own `n` after the nested call returns.

P9.

```
module p9;
  task show(input int v);
    #1 $display("%0d", v);
  endtask
  initial fork
    show(1);
    show(2);
    show(3);
  join
endmodule
```

Answer: the same number three times; `3 3 3` on simulators that start the branches in source order. The task is static, so all three calls share `v`, and the last call to start overwrites it before any of them prints.

P10.

```
module p10;
  function automatic void f();
    fork
      #5 $display("B %0t", $time);
    join_none
    $display("A %0t", $time);
  endfunction
  initial begin
    f();
    $display("C %0t", $time);
    #10 $display("D %0t", $time);
  end
endmodule
```

Answer: `A 0`, `C 0`, `B 5`, `D 10`. The function returns in zero time; the forked statement runs in the background.

P11.

```
module p11;
  int v;
  task automatic t(output int o);
    o = 1;
    #10;
    o = 2;
  endtask
  initial begin
    fork
      t(v);
    join_none
    #5  $display("%0d", v);
    #10 $display("%0d", v);
  end
endmodule
```

Answer: `0` then `2`. At time 5 the task is still running, and an output argument is copied back only at the return (time 10). The value 1 is never seen by the caller.

P12.

```
module p12;
  int x;
  function automatic int twice(int v);
    x = x + 1;
    return 2 * v;
  endfunction
  initial begin
    x = 10;
    x = twice(x) + twice(x);
    $display("%0d", x);
  end
endmodule
```

Answer: `42` if the operands are evaluated left to right, which is what simulators do: the first call receives 10, sets `x` to 11 and returns 20; the second receives 11, sets `x` to 12 and returns 22; the sum 42 is then assigned to `x`. The standard does not fix the evaluation order of operands, so a function with side effects on a variable used in the same expression is a bug to avoid, not a feature to rely on.

6.4 Find-the-Error and Debugging Questions

Compile-time errors

E1.

```
function automatic int delayed_add(int a, int b);
  #1;
  return a + b;
endfunction
```

A delay in a function. Remove the delay, or make it a task with an `output` argument.

E2.

```
task automatic log_it(string s);
  $display("%s", s);
endtask
function automatic int f(int a);
  log_it("in f");
  return a;
endfunction
```

A function calling a task. Declare `log_it` as `function automatic void`.

E3.

```
module m;
  task incr(ref int x);
    x++;
  endtask
endmodule
```

`ref` in a static task. Write `task automatic incr(ref int x);`.

E4.

```
task automatic incr(ref int x);
  x++;
endtask
int a;
initial begin
  incr(5);
  incr(a + 1);
end
```

A `ref` actual must be a variable. A constant and an expression cannot be passed by reference. Call `incr(a);`.

E5.

```
task automatic t();
  int x;
  x <= 1;
endtask
```

A nonblocking assignment to an automatic variable. Use `x = 1;`.

E6.

```
task automatic get_val(output int v);
  v = 42;
endtask
int y;
initial y = get_val(y);
```

A task used in an expression. Call it as a statement: `get_val(y);`.

E7.

```
function automatic void f();
  return 1;
endfunction
task automatic t();
  return 5;
endtask
```

A void function and a task cannot return a value. Use `return;`, or give the function a return type.

E8.

```
function automatic void reset_all(const ref int arr[]);
  foreach (arr[i]) arr[i] = 0;
endfunction
```

Assignment to a `const ref` argument. Remove `const`.

E9.

```
function automatic void incr(ref int x);
  x++;
endfunction
bit [7:0] b;
wire [31:0] w;
initial begin
  incr(b);
  incr(w);
end
```

Two errors. The type of a `ref` actual must match the formal exactly (`bit [7:0]` is not `int`), and a net cannot be passed by reference.

E10.

```
function automatic int f(int a, int b = 2, int c);
  return a + b + c;
endfunction
initial $display("%0d", f(1));
```

`c` has no default and is not supplied. Call `f(1, , 3)` or give `c` a default.

E11.

```
function automatic void scale(ref int arr[], int factor);
  foreach (arr[i]) arr[i] *= factor;
endfunction
int d[] = '{1, 2, 3};
initial scale(d, 10);
```

`factor` inherits `ref` from the previous argument, so the literal 10 cannot be passed. Declare it `input int factor`.

E12.

```
class c;
  function new();
    @(posedge tb.clk);
  endfunction
endclass
```

The constructor is a function and cannot contain an event control. Move the wait into a task of the class.

Run-time bugs (the code compiles)

D1.

```
task write(input bit [7:0] addr, input bit [7:0] data);
  @(posedge clk);
  bus_addr <= addr;
  bus_data <= data;
endtask
initial fork
  write(8'h10, 8'hAA);
  write(8'h20, 8'hBB);
join
```

The task is static. The second call overwrites `addr` and `data` while the first is waiting for the clock, so one transfer is lost. Declare the task `automatic`, and add a semaphore so that the two transfers do not use the bus in the same cycle.

D2.

```
task automatic wait_flag(input bit flag);
  wait (flag);
endtask
```

Hangs if `flag` is 0 at the call: the task waits on a private copy. Use `ref bit flag`.

D3.

```
task automatic ramp(output logic [7:0] level);
  for (int i = 0; i < 8; i++) begin
    @(posedge clk);
    level = i;
  end
endtask
initial ramp(dut_level);
```

`dut_level` does not ramp. It is written once, with the final value 7, when the task returns. Use a `ref` argument, or assign the signal directly.

D4.

```
always begin
  if (en) count++;
end
```

No timing control: a zero-delay infinite loop at time 0. Add `@(posedge clk)`.

D5.

```
task check_burst();
  int errors = 0;
  // ... counts errors in this burst ...
  if (errors > 0) $display("%0d errors in burst", errors);
endtask
```

In a static task `errors` is initialised once, so the count accumulates over all bursts. (Strict tools also reject the declaration because the lifetime keyword is missing.) Declare the task `automatic`.

D6.

```
function automatic checksum(bit [7:0] d[]);
  int s;
  s = 0;
  foreach (d[i]) s += d[i];
  return s;
endfunction
```

No return type: the checksum is truncated to one bit. Declare `function automatic int checksum(...)`. The array should also be `const ref`.

D7.

```
function int depth(int n);
  if (n <= 1) return 0;
  return 1 + depth(n / 2);
endfunction
```

A recursive function without `automatic`. The result may be wrong and tools warn. Add `automatic`.

D8.

```
task automatic drive(input logic [7:0] a, input logic [7:0] b);
  @(posedge clk);
  a = a;
  b = b;
endtask
```

The arguments have the same names as the signals the task should drive, so the assignments copy each argument to itself and the signals never change. Rename the arguments (`a_i`, `b_i`) or drive through a hierarchical or interface reference.

D9.

```
task automatic wait_done();
  @(posedge done);
endtask
```

If `done` is already high when the task is called, the task waits for the next rising edge, possibly for ever. Use `wait (done);`.

D10.

```
task automatic send(input logic [7:0] d);
  @(posedge clk);
  data  = d;
  valid = 1;
endtask
```

Blocking assignments at the active clock edge race with the DUT, which samples on the same edge. Use nonblocking assignments, drive on the opposite edge, or use a clocking block.

6.5 Coding Exercises with Solutions

Exercise 1: reverse the bits of a byte

Write a function that returns its 8-bit argument with the bit order reversed.

ex01_reverse_bits.sv

```
module ex01_reverse_bits;

  function automatic bit [7:0] reverse8(bit [7:0] d);
    bit [7:0] r;
    for (int i = 0; i < 8; i++) r[i] = d[7 - i];
    return r;
  endfunction

  initial begin
    $display("%b -> %b", 8'b1011_0001, reverse8(8'b1011_0001));
    $display("%b -> %b", 8'b0000_0001, reverse8(8'b0000_0001));
  end

endmodule
```

EXPECTED OUTPUT

```
10110001 -> 10001101
00000001 -> 10000000
```

Exercise 2: ceiling of log2

Write a function that returns the number of address bits needed for a memory of `n` locations (the ceiling of log2 `n`), without using `$clog2`.

ex02_clog2.sv

```
module ex02_clog2;

  function automatic int clog2(int unsigned n);
    int r;
    r = 0;
    while ((1 << r) < n) r++;
    return r;
  endfunction

  initial begin
    $display("clog2(1)=%0d clog2(2)=%0d clog2(16)=%0d clog2(17)=%0d clog2(1000)=%0d",
             clog2(1), clog2(2), clog2(16), clog2(17), clog2(1000));
  end

endmodule
```

EXPECTED OUTPUT

```
clog2(1)=0 clog2(2)=1 clog2(16)=4 clog2(17)=5 clog2(1000)=10
```

The loop finds the smallest `r` for which 2^r is at least `n`. The function has no side effects and its result depends only on its argument, so it could also be used to compute a `localparam`.

Exercise 3: swap two variables

Write a task that swaps the values of two `int` variables of the caller.

ex03_swap.sv

```
module ex03_swap;

  task automatic swap(ref int x, ref int y);
    int tmp;
    tmp = x;
    x   = y;
    y   = tmp;
  endtask

  int a, b;

  initial begin
    a = 3;
    b = 7;
    swap(a, b);
    $display("a=%0d b=%0d", a, b);
  end

endmodule
```

EXPECTED OUTPUT

```
a=7 b=3
```

`inout` arguments would also work here. `input` arguments would not: the task would swap its own copies. Because nothing in it consumes time, `swap` would be better declared as a void function; it is written as a task to show that `ref` works in both.

Exercise 4: several results from one function

Write a function that computes the minimum, maximum and average of an array of any size, without copying the array.

ex04_array_stats.sv

```
module ex04_array_stats;

  function automatic void stats(const ref int arr[],
                                output    int lo,
                                output    int hi,
                                output    real avg);
    int sum;
    lo  = arr[0];
    hi  = arr[0];
    sum = 0;
    foreach (arr[i]) begin
      if (arr[i] < lo) lo = arr[i];
      if (arr[i] > hi) hi = arr[i];
      sum += arr[i];
    end
    avg = real'(sum) / arr.size();
  endfunction

  int  values[] = '{4, 9, 2, 7};
  int  lo, hi;
  real avg;

  initial begin
    stats(values, lo, hi, avg);
    $display("min=%0d max=%0d avg=%0.2f", lo, hi, avg);
  end

endmodule
```

EXPECTED OUTPUT

```
min=2 max=9 avg=5.50
```

`const ref` avoids the copy. Each `output` has its direction written explicitly; otherwise the arguments after `arr` would inherit `const ref`. The cast `real'(sum)` forces a real division (Module 1, Section 2.7): 22 / 4 as integers would give 5.

Exercise 5: recursive palindrome check

Write a recursive function that reports whether a string reads the same forwards and backwards.

ex05_palindrome.sv

```
module ex05_palindrome;

  function automatic bit is_pal_range(string s, int lo, int hi);
    if (lo >= hi)       return 1;                // base case: 0 or 1 characters left
    if (s[lo] != s[hi]) return 0;                // base case: mismatch
    return is_pal_range(s, lo + 1, hi - 1);      // recursive case: the inner part
  endfunction

  function automatic bit is_palindrome(string s);
    return is_pal_range(s, 0, s.len() - 1);
  endfunction

  initial begin
    $display("level -> %b", is_palindrome("level"));
    $display("hello -> %b", is_palindrome("hello"));
    $display("noon  -> %b", is_palindrome("noon"));
  end

endmodule
```

EXPECTED OUTPUT

```
level -> 1
hello -> 0
noon  -> 1
```

The recursion has two base cases and moves both indices towards the middle on each call. `s[i]` is the i-th character of the string as a byte, and `s.len()` is the number of characters.

Exercise 6: a unique-ID generator

Write a function that returns 0, 1, 2... on successive calls, without using a module-level variable.

ex06_next_id.sv

```
module ex06_next_id;

  function automatic int unsigned next_id();
    static int unsigned id = 0;     // one copy, initialised once
    id++;
    return id - 1;
  endfunction

  initial begin
    $display("%0d", next_id());
    $display("%0d", next_id());
    $display("%0d", next_id());
  end

endmodule
```

EXPECTED OUTPUT

```
0
1
2
```

A deliberate use of a static variable inside an automatic function. The function increments the stored value and returns the value it had before. Each `$display` contains one call, so the order of the calls is certain.

Exercise 7: serial send and receive tasks

Write a task `send_serial` that sends a byte one bit per clock (LSB first) on `sdata`, with `svalid` high while the bits are valid, and a task `recv_serial` that receives it. Run them in parallel and check that the byte arrives.

ex07_serial.sv

```
module ex07_serial;

  logic clk    = 0;
  logic sdata  = 0;
  logic svalid = 0;

  always #5 clk = ~clk;

  task automatic send_serial(input bit [7:0] data);
    for (int i = 0; i < 8; i++) begin
      @(negedge clk);              // drive on the falling edge
      svalid = 1'b1;
      sdata  = data[i];
    end
    @(negedge clk);
    svalid = 1'b0;
  endtask

  task automatic recv_serial(output bit [7:0] data);
    int i;
    i = 0;
    while (i < 8) begin
      @(posedge clk);              // sample on the rising edge
      if (svalid) begin
        data[i] = sdata;
        i++;
      end
    end
  endtask

  bit [7:0] rx;

  initial begin
    fork
      send_serial(8'hA5);
      recv_serial(rx);
    join
    $display("[%0t] sent a5, received %h", $time, rx);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[90] sent a5, received a5
```

The sender drives the eight bits at the falling edges 10, 20, ... 80 and removes `svalid` at 90. The receiver samples at the rising edges 15, 25, ... 85, each in the middle of a bit, and returns at 85; its `output` argument is copied to `rx` at that moment. The `fork`...`join` ends when both tasks have returned, at 90. Driving and sampling on opposite edges avoids any race between the two tasks. The receiver first samples at time 5, sees `svalid` = 0 and ignores that edge.

Exercise 8: recursive conversion to a binary string

Write a recursive function that converts an unsigned number to a string of binary digits with no leading zeros.

ex08_to_bin.sv

```
module ex08_to_bin;

  function automatic string to_bin(int unsigned n);
    string digit;
    digit = (n % 2) ? "1" : "0";
    if (n < 2) return digit;                  // base case: a single digit
    return {to_bin(n / 2), digit};            // higher digits first, then this one
  endfunction

  initial begin
    $display("0   -> %s", to_bin(0));
    $display("10  -> %s", to_bin(10));
    $display("255 -> %s", to_bin(255));
  end

endmodule
```

EXPECTED OUTPUT

```
0   -> 0
10  -> 1010
255 -> 11111111
```

`{a, b}` with string operands concatenates strings. Each level contributes its own least significant digit after the digits produced by the deeper levels.

Further practice (no solutions given)

1. Write `function automatic int count_ones(const ref bit [7:0] bytes[]);` that returns the total number of 1 bits in an array of bytes.
2. Write a task `pulse(ref logic sig, input int cycles = 1)` that sets `sig` high for the given number of clock cycles and then low. Explain why `output` would not work.
3. Rewrite `fib` without recursion and compare the number of iterations with the number of recursive calls for `n` = 20.
4. Extend `tb_alu_tasks` with the three operations that are not exercised (`ALU_OR`, `ALU_XOR`, `ALU_SHR`) and with a deliberately wrong expected value, and confirm that `check` reports the failure.
5. Write a function `alu_op_e random_op()` that returns a random opcode using `$urandom_range` and a cast. Which kind of cast is appropriate, and why?
6. Write a recursive function `power(base, exp)` that uses at most about log2(`exp`) multiplications.
7. Predict, then run: a static task with a `for (int i = 0; i < 3; i++) #1;` loop and a message at the end printing an input argument, called from three `fork` branches with different arguments.
8. Take any task from this module that contains no timing control and convert it to a void function. Then call it from another function.

6.6 Concepts You Must Understand Before Moving Forward

Procedural code

- [ ] I can explain the difference between `initial`, `always` and `final`, and why an `always` block needs a timing control.
- [ ] I can explain blocking and nonblocking assignments and predict the result of a swap written each way.

- [ ] I can list the main event regions in order and say where `$display`, `$strobe` and nonblocking updates happen.
- [ ] I know what `#0` does and why it does not reveal a nonblocking update.
- [ ] I can explain the difference between `@(posedge sig)` and `wait (sig)`.

Functions and tasks

- [ ] I can state at least four differences between a function and a task, starting with time.
- [ ] I can explain why a function cannot consume time, using expressions as the reason.
- [ ] I know the default return type of a function and the three ways a result is produced.
- [ ] I know when to use a void function instead of a task.
- [ ] I know who can call whom: function → function, task → both, function → task never (except `fork`...`join_none`).

Arguments

- [ ] I can describe copy-in and copy-out for `input`, `output` and `inout`.
- [ ] I know that an `output` formal starts at the type's default and is copied back only at the return.
- [ ] I can explain `ref` and `const ref`, their restrictions, and when each is needed.
- [ ] I can explain what happens when an array, a struct and a class handle are passed as `input`.
- [ ] I can use default values, empty positions and named binding.
- [ ] I know how direction and type are inherited by following arguments.

Lifetime and recursion

- [ ] I know the default lifetime in modules, packages and classes.
- [ ] I can reproduce and explain the shared-variable bug of a static task called from two threads.
- [ ] I know the explicit-keyword rule for initialised variables in static contexts.
- [ ] I can write a recursive function with a base case and explain why it must be automatic.

Classes (preview)

- [ ] I know that methods are functions and tasks with automatic lifetime, and that `new` is a function.
- [ ] For each component of the layered testbench I can say which methods are tasks and which are functions.

6.7 What Comes Next

Module 4 covers classes and object-oriented programming: declaring classes, creating objects with `new`, handles and null handles, `this`, static properties and methods, access control, shallow and deep copy, inheritance, `super`, virtual methods, polymorphism, abstract classes and `$cast` between handles. The functions and tasks of this module become the methods of those classes, with the argument-passing and lifetime rules unchanged. The `alu_txn` class previewed in Section 5.4 is developed there in full.